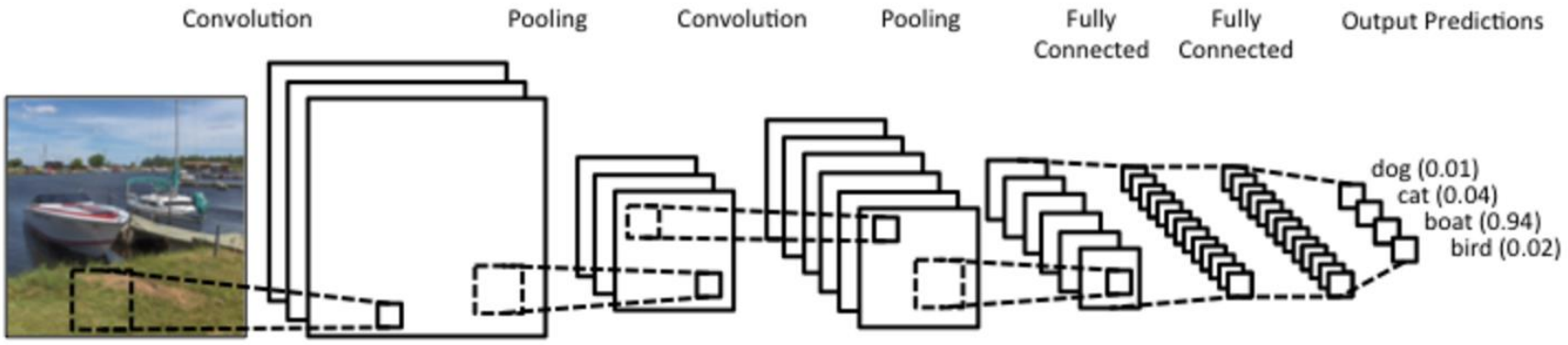
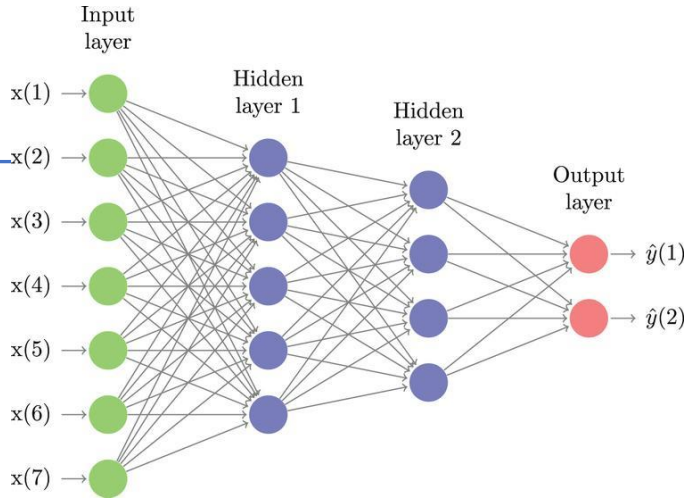
Konvolucione neuronske mreže

Convolutional Neural Networks (CNN)



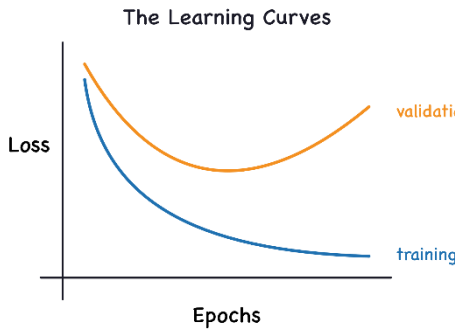
Zašto CNN a ne NN?

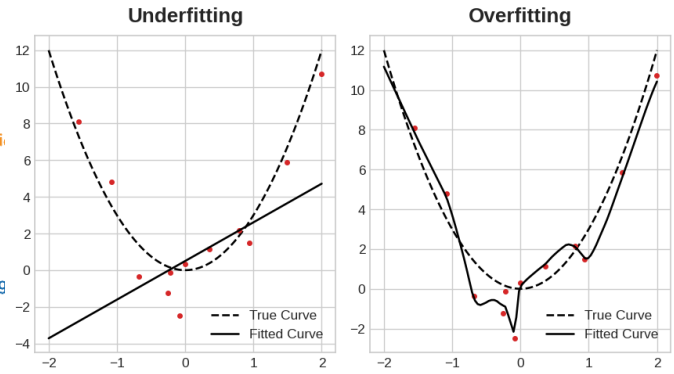


- Slika: ogroman broj ulaznih obeležja
  - Rezolucija $64 \times 64 \rightarrow 64 \cdot 64 \cdot 3 = 12\,228$
  - Rezolucija $1000 \times 1000 \rightarrow 3 \cdot 10^6$

- Potpuno povezana neuronska mreža: neka prvi skriveni sloj sadrži 1000 neurona. Koliko parametara moramo naučiti za taj sloj?

Overfitting





Računarska složenost

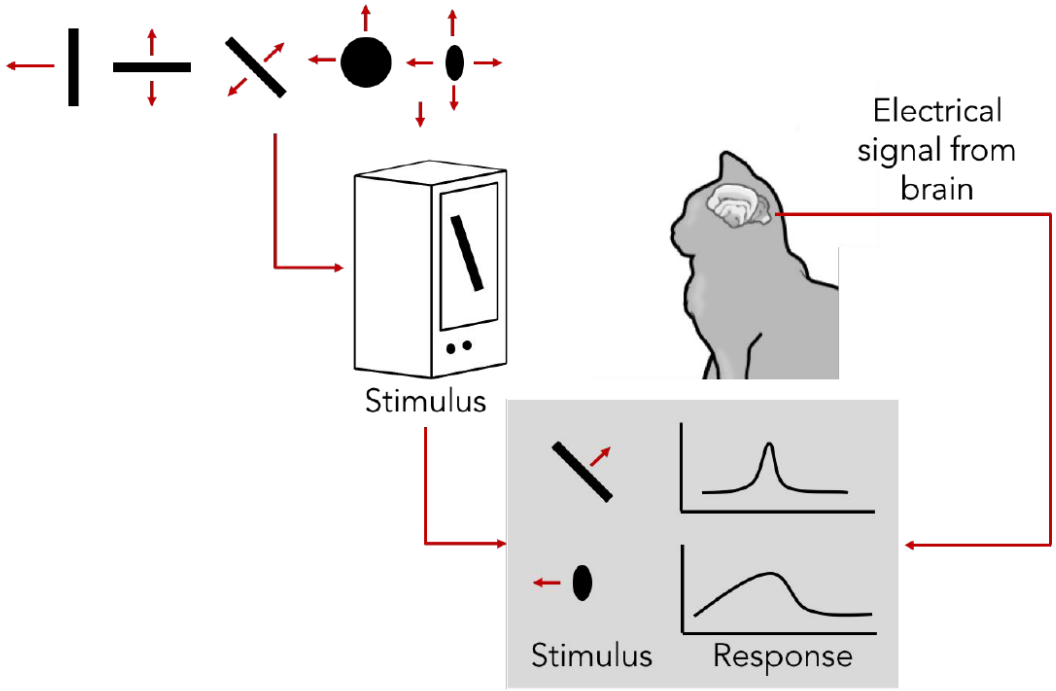
Memorijski resursi

Ne želimo da budemo osuđeni na slike niske rezolucije!
Moramo smanjiti broj podesivih parametara.

Inspiracija za CNN

A bit of history:

Hubel & Wiesel,
1959
RECEPTIVE FIELDS OF SINGLE NEURONES IN
THE CAT'S STRIATE CORTEX
1962
RECEPTIVE FIELDS, BINOCULAR INTERACTION
AND FUNCTIONAL ARCHITECTURE IN
THE CAT'S VISUAL CORTEX
1968...



Cat image by CNX OpenStax is licensed under CC BY 4.0; changes made

Hijerarhijska organizacija

Hierarchical organization

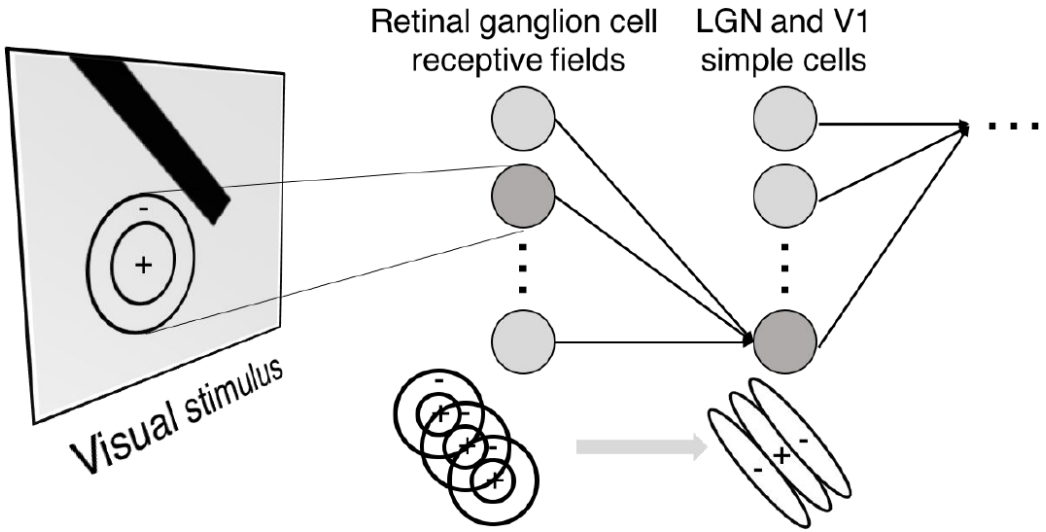
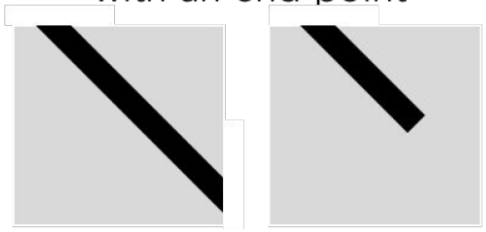


Illustration of hierarchical organization in early visual pathways by Lane McIntosh, copyright CS231n 2017

Simple cells:
Response to light orientation

Complex cells:
Response to light orientation and movement

Hypercomplex cells:
response to movement with an end point



No response

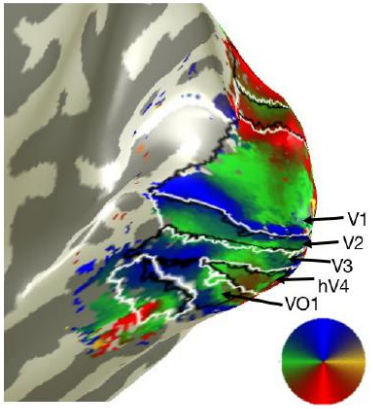
Response (end point)

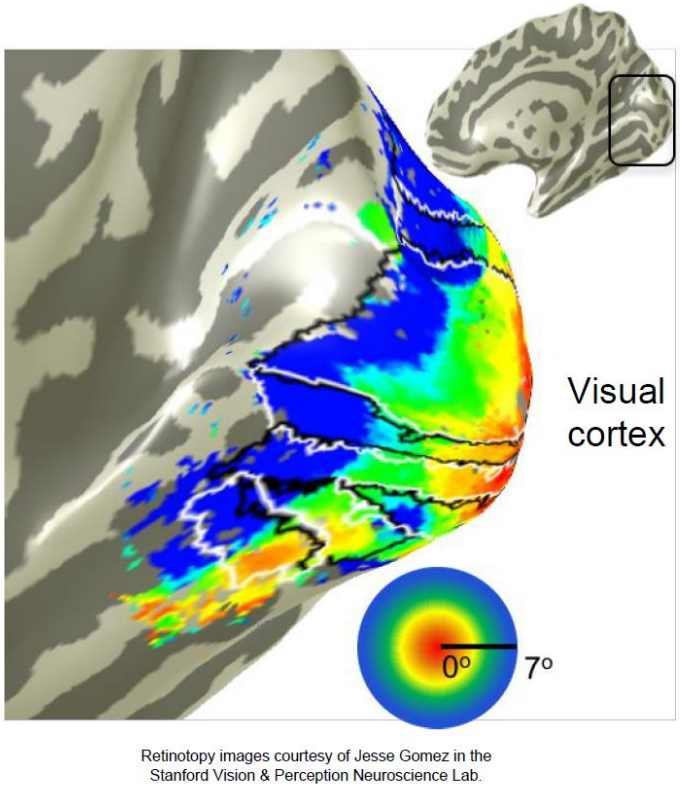
Topologija

A bit of history

Topographical mapping in the cortex:

ćelije koje su blizu u korteksu procesiraju regione koji su blizu u vizuelnom polju





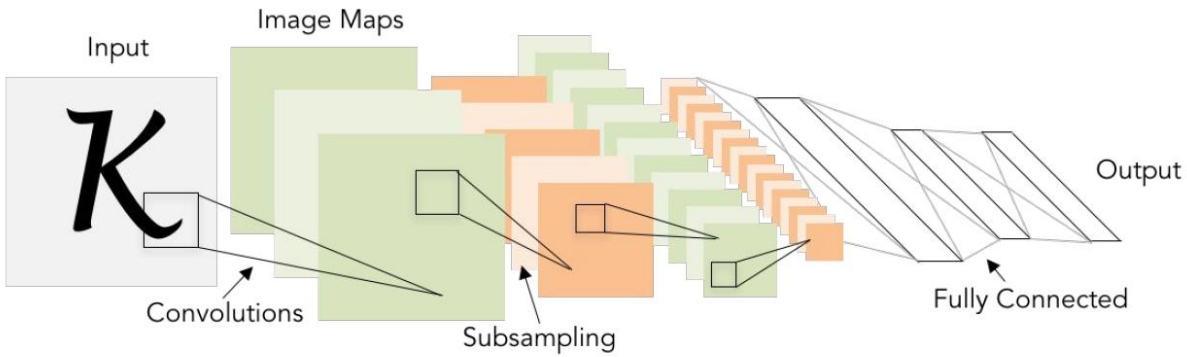
Retinotopy images courtesy of Jesse Gomez in the Stanford Vision & Perception Neuroscience Lab.

CNN istorija

Prvi primer primene backpropagation i gradijent-baziranog učenja za treniranje CNN

document recognition
[LeCun, Bottou, Bengio, Haffner 1998]

Ali još nije dobro skaliralo na izazovnije i kompleksnije podatke...



LeNet-5

CNN istorija

- Prva moderna inkarnacija CNN
- Ne razlikuje se u značajnoj meri od CNN sa kojom je radio LeCun
- Ali je dublja i imala je mogućnost da iskoristi veće količine podataka kao i paralelnog računanja u GPU

[Krizhevsky, Sutskever, Hinton, 2012]

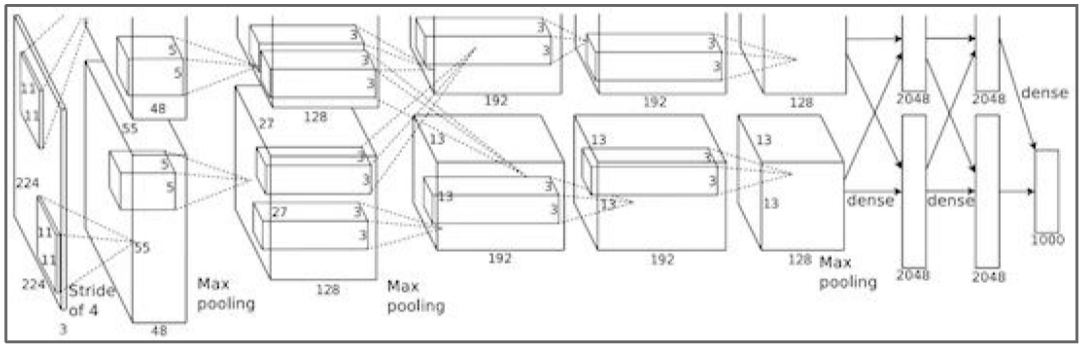
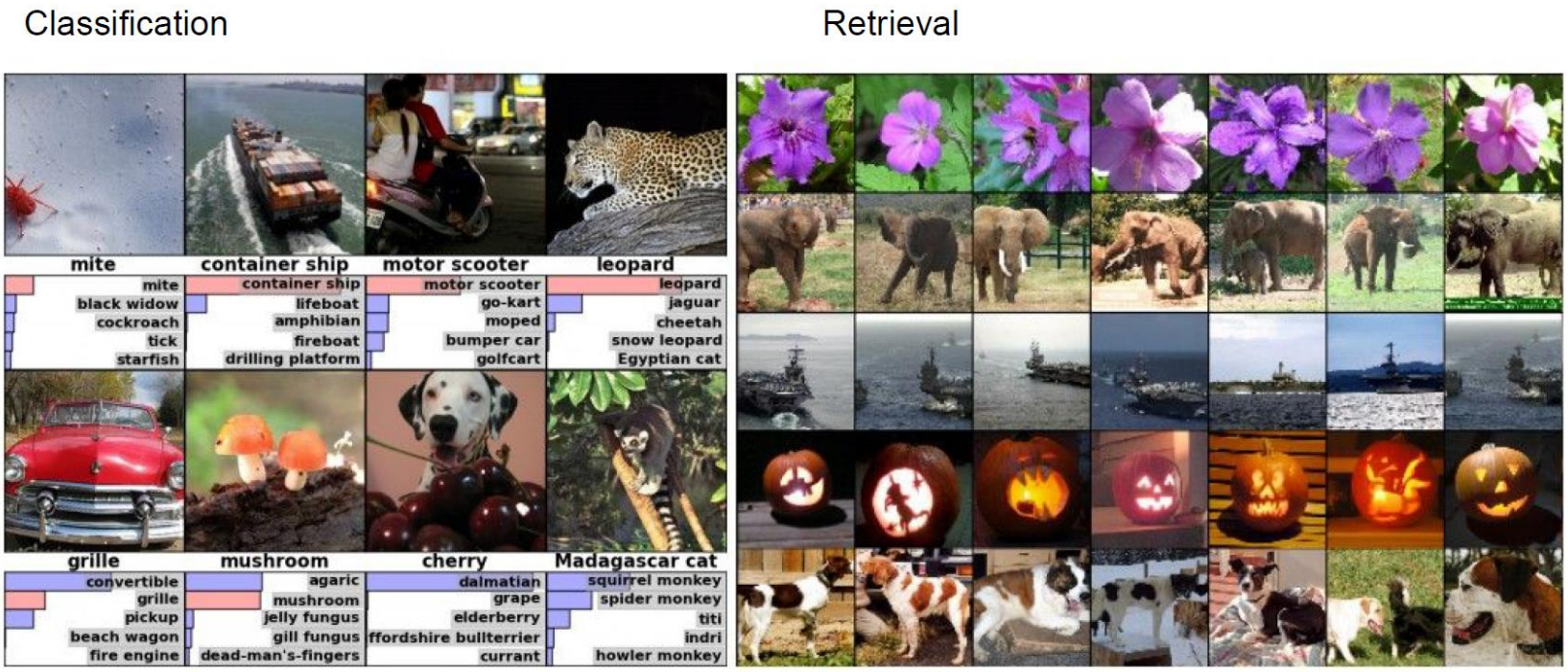


Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

"AlexNet"

CNN danas

Klasifikacija i dobavljanje slika

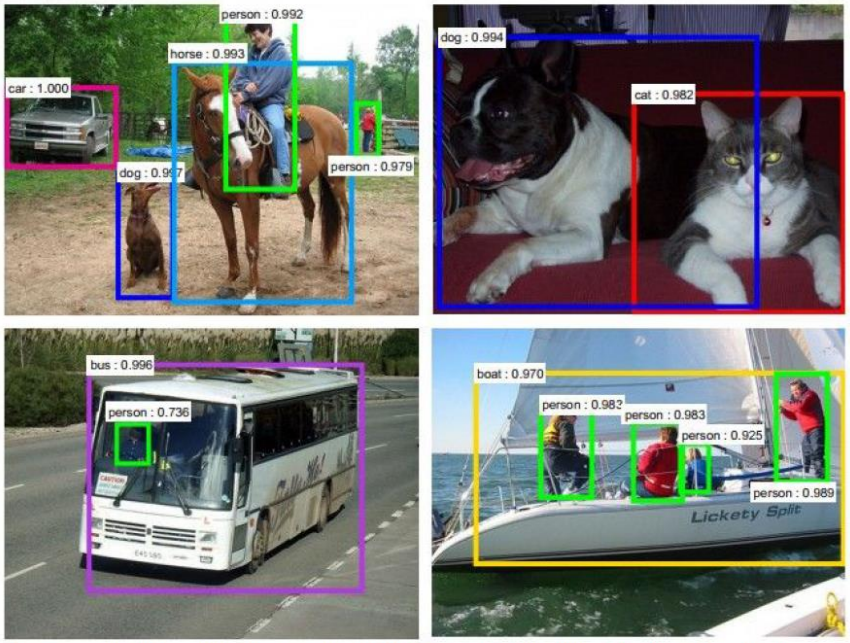


Figures copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

CNN danas

Lokalizacija (gde se objekat nalazi na slici) i segmentacija

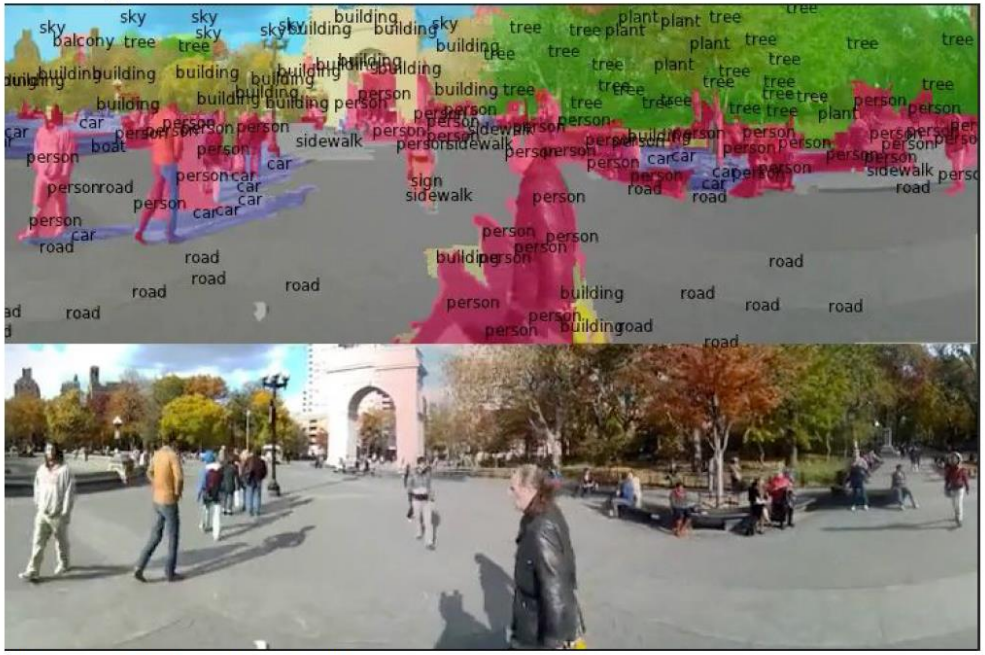
Detection



Figures copyright Shaoqing Ren, Kaiming He, Ross Girschick, Jian Sun, 2015. Reproduced with permission.

[Faster R-CNN: Ren, He, Girshick, Sun 2015]

Segmentation



Figures copyright Clement Farabet, 2012. Reproduced with permission.

[Farabet et al., 2012]

CNN danas

Samovozeća kola

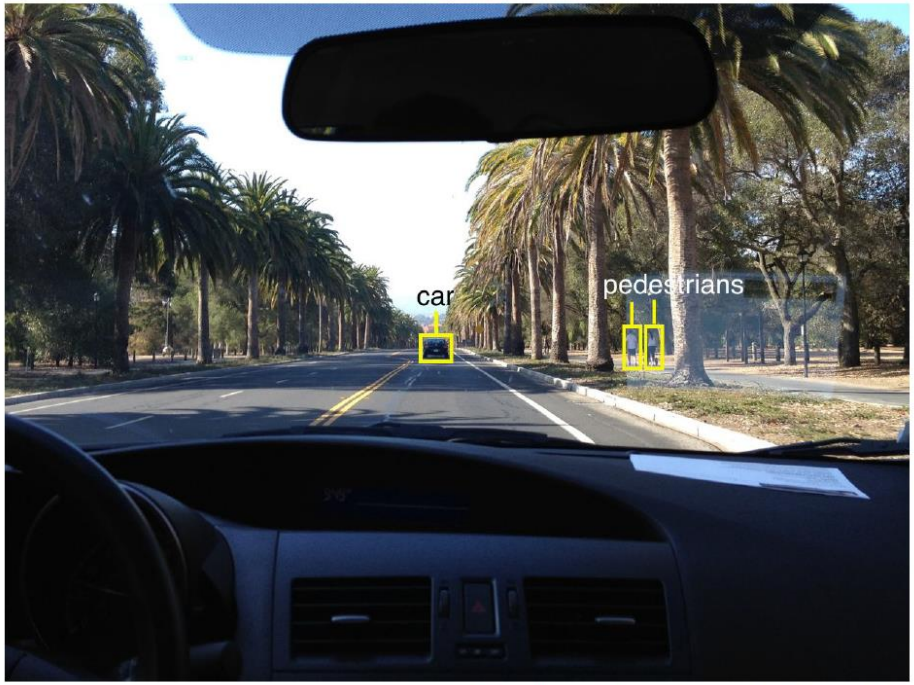


Photo by Lane McIntosh. Copyright CS231n 2017.

self-driving cars



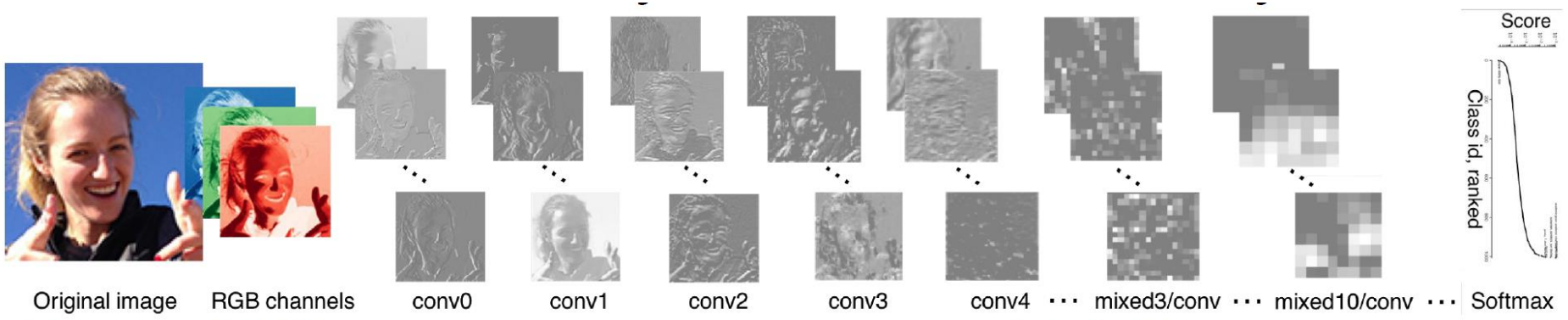
This image by GBPublic_PR is licensed under CC-BY 2.0

NVIDIA Tesla line
(these are the GPUs on rye01.stanford.edu)

Note that for embedded systems a typical setup would involve NVIDIA Tegras, with integrated GPU and ARM-based CPU cores.

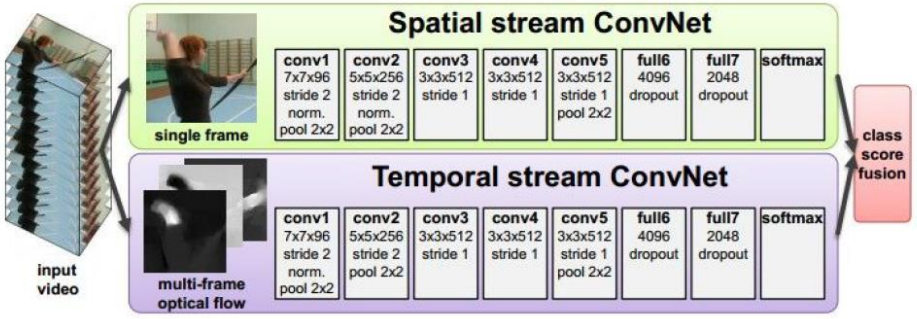
CNN danas

Prepoznavanje lica, klasifikacija videa



[Taigman et al. 2014]

Activations of inception-v3 architecture [Szegedy et al. 2015] to image of Emma McIntosh, used with permission. Figure and architecture not from Taigman et al. 2014.



[Simonyan et al. 2014]

Figures copyright Simonyan et al., 2014. Reproduced with permission.

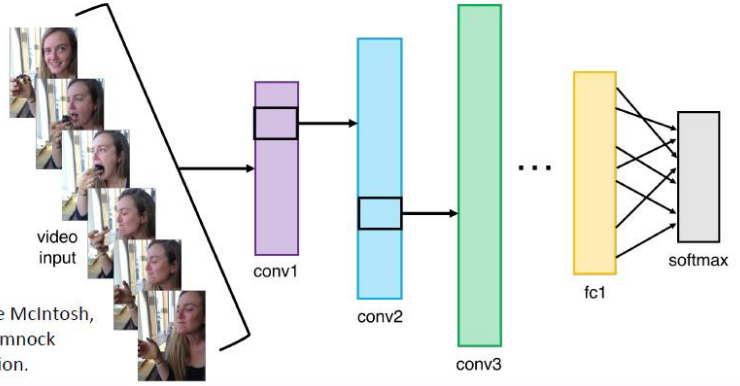


Illustration by Lane McIntosh, photos of Katie Cumnock used with permission.

CNN danas

Prepoznavanje poze, igranje igara



Images are examples of pose estimation, not actually from Toshev & Szegedy 2014. Copyright Lane McIntosh.

[Toshev, Szegedy 2014]

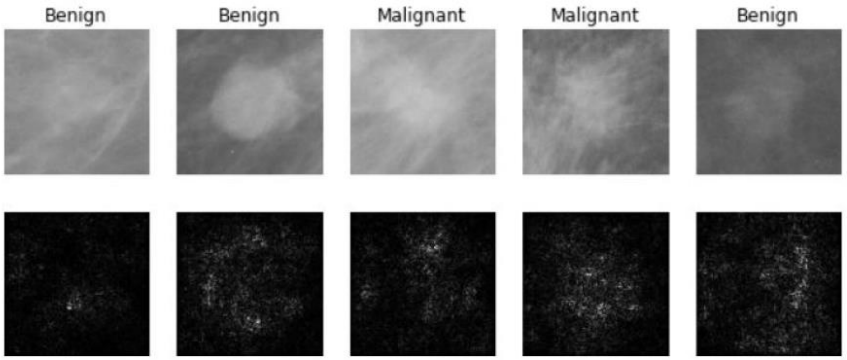


[Guo et al. 2014]

Figures copyright Xiaoxiao Guo, Satinder Singh, Honglak Lee, Richard Lewis, and Xiaoshi Wang, 2014. Reproduced with permission.

CNN danas

Interpretacija i dijagnoza medicinskih slika, klasifikacija galaksija, prepoznavanje saobraćajnih znakova



[Levy et al. 2016]

Figure copyright Levy et al. 2016.
Reproduced with permission.



[Dieleman et al. 2014]

From left to right: public domain by NASA, usage permitted by ESA/Hubble, public domain by NASA, and public domain.



[Sermanet et al. 2011]
[Ciresan et al.]

Photos by Lane McIntosh.
Copyright CS231n 2017.

CNN danas

This image by Christin Khan is in the public domain and originally came from the U.S. NOAA.



Whale recognition, Kaggle Challenge

Photo and figure by Lane McIntosh; not actual example from Mnih and Hinton, 2010 paper.



Mnih and Hinton, 2010

CNN danas

No errors



A white teddy bear sitting in the grass



A man riding a wave on top of a surfboard

Minor errors



A man in a baseball uniform throwing a ball

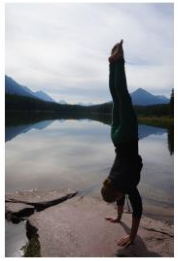


A cat sitting on a suitcase on the floor

Somewhat related



A woman is holding a cat in her hand



A woman standing on a beach holding a surfboard

Image Captioning

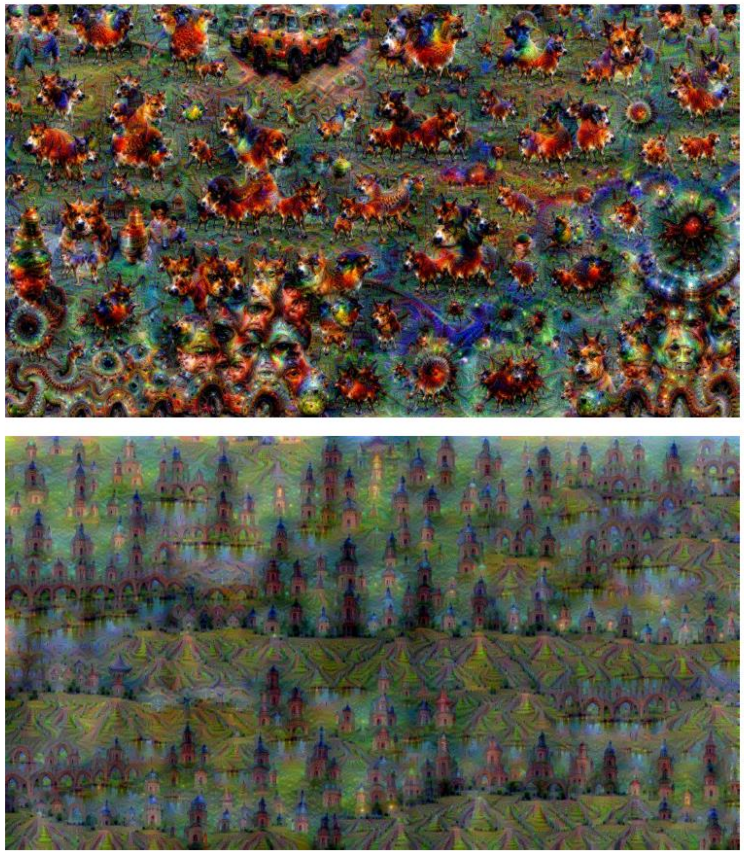
[Vinyals et al., 2015]
[Karpathy and Fei-Fei, 2015]

All images are CC0 Public domain:
https://pixabay.com/en/luggage-antique-cat-1643010/
https://pixabay.com/en/teddy-plush-bears-cute-teddy-bear-1623436/
https://pixabay.com/en/surf-wave-summer-sport-litoral-1668716/
https://pixabay.com/en/woman-female-model-portrait-adult-983967/
https://pixabay.com/en/handstand-lake-meditation-496008/
https://pixabay.com/en/baseball-player-shortstop-infield-1045263/

Captions generated by Justin Johnson using Neuraltalk2

CNN danas

Umetnička dela, transfer stila



Figures copyright Justin Johnson, 2015. Reproduced with permission. Generated using the Inceptionism approach from a blog post by Google Research.

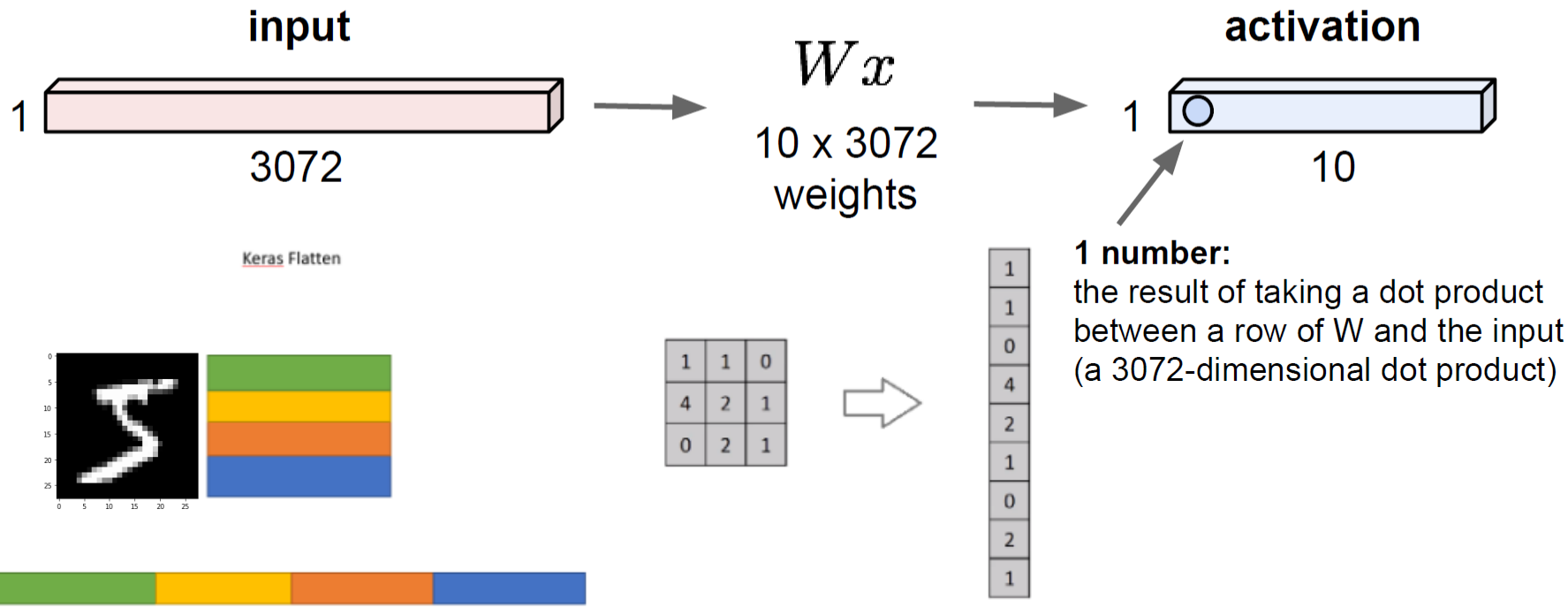


Original image is CC0 public domain
Starry Night and Tree Roots by Van Gogh are in the public domain
Bokeh image is in the public domain
Stylized images copyright Justin Johnson, 2017; reproduced with permission

Gatys et al, "Image Style Transfer using Convolutional Neural Networks", CVPR 2016
Gatys et al, "Controlling Perceptual Factors in Neural Style Transfer", CVPR 2017

Potpuno povezan sloj

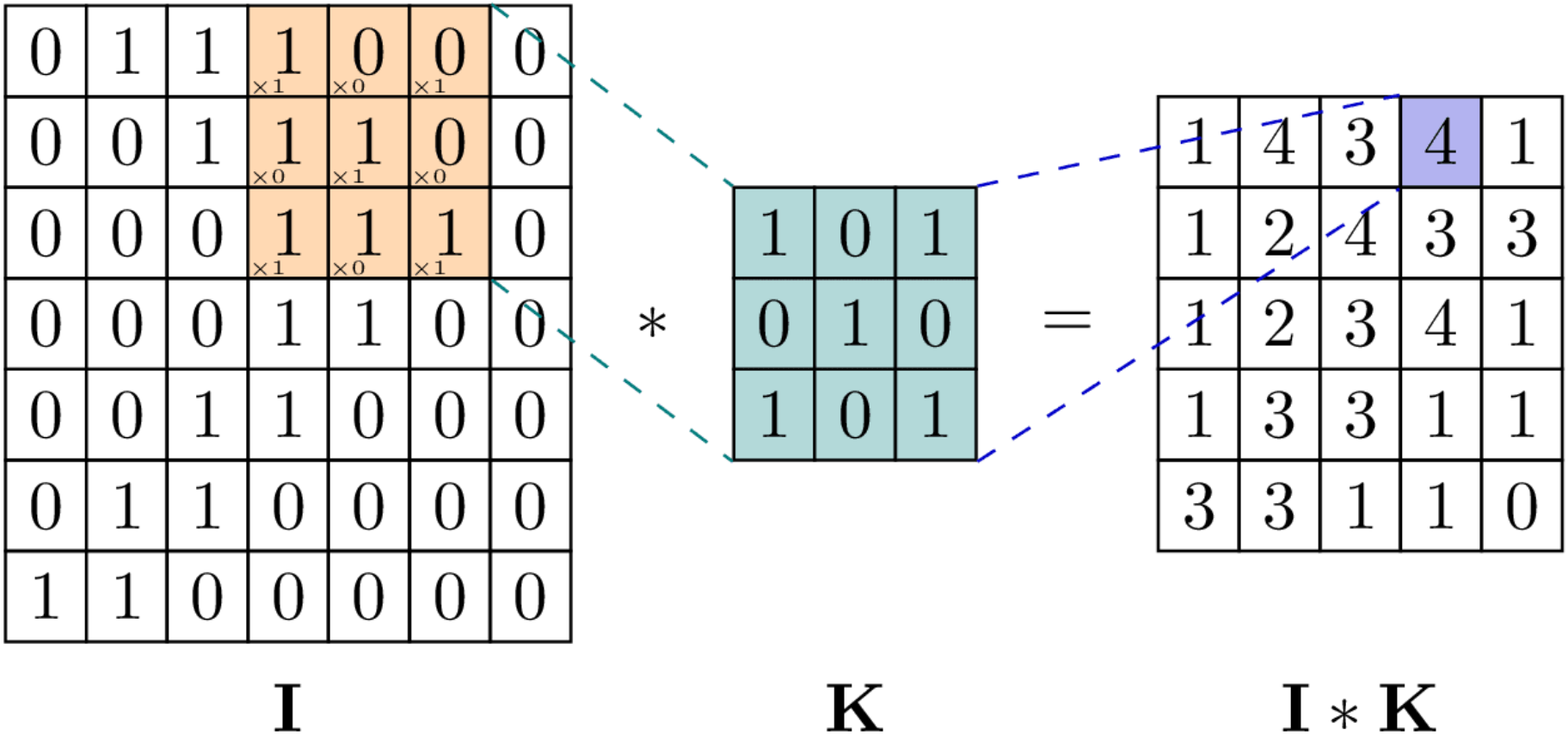
32x32x3 image -> stretch to 3072 x 1



Rezultat će biti niz od 28 x 28 = 784 elemenata:

```
[ ]  x_train = x_train.reshape(60000, 784)
     x_valid = x_valid.reshape(10000,784)
```

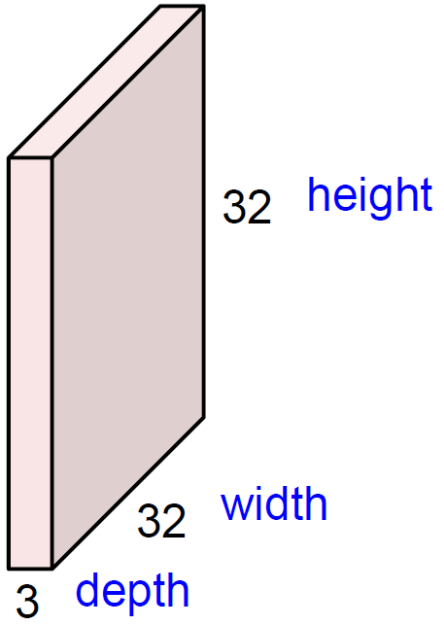
Operacija konvolucije



Konvolucioni sloj

Convolution Layer

32x32x3 image -> preserve spatial structure

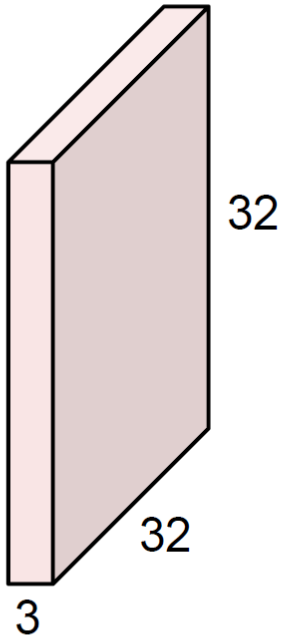


Konvolucioni sloj

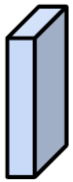
Convolution Layer

Filters always extend the full depth of the input volume

32x32x3 image



5x5x3 filter

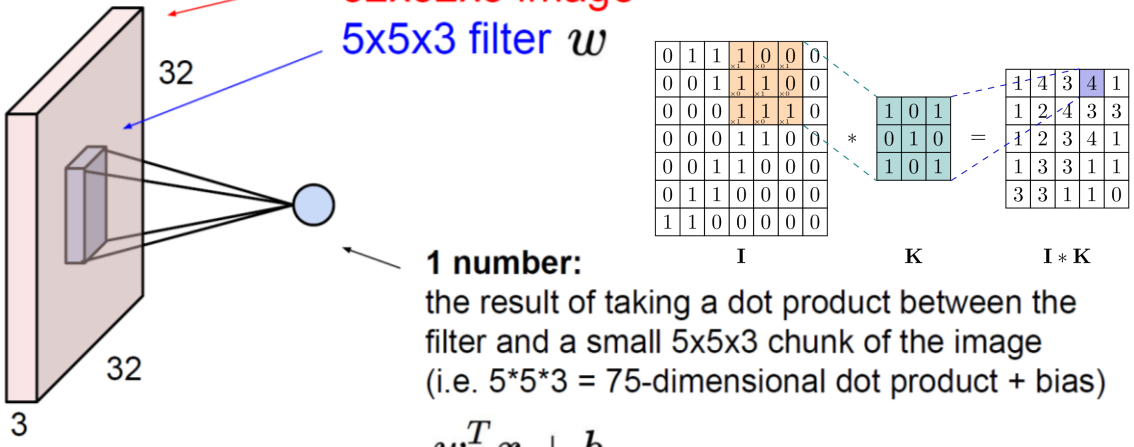


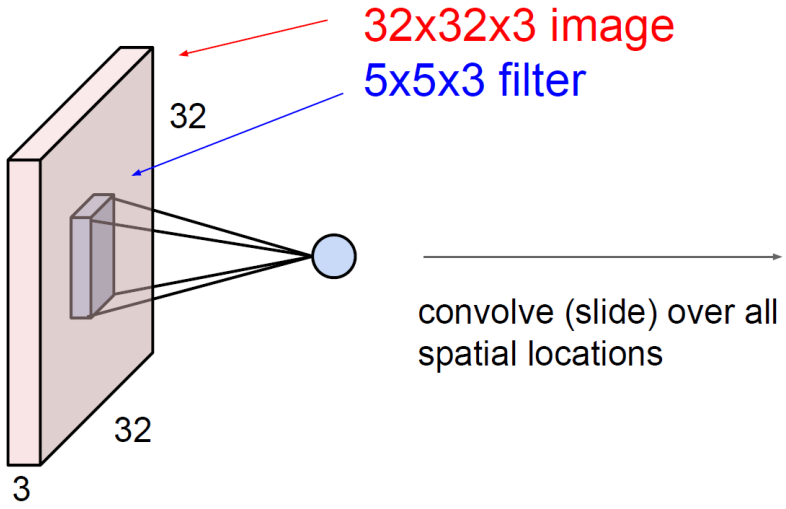
Težine koje treba da naučimo

Convolve the filter with the image i.e. “slide over the image spatially, computing dot products”

Glavna razlika u odnosu na potpuno povezan sloj jeste zadržavanje prostorne strukture

Konvolucioni sloj





Konvolucioni sloj

2	2	4	4	0
80	60	10	7	10
4	10	80	10	20
12	24	10	8	20
22	42	20	10	10

Image

X

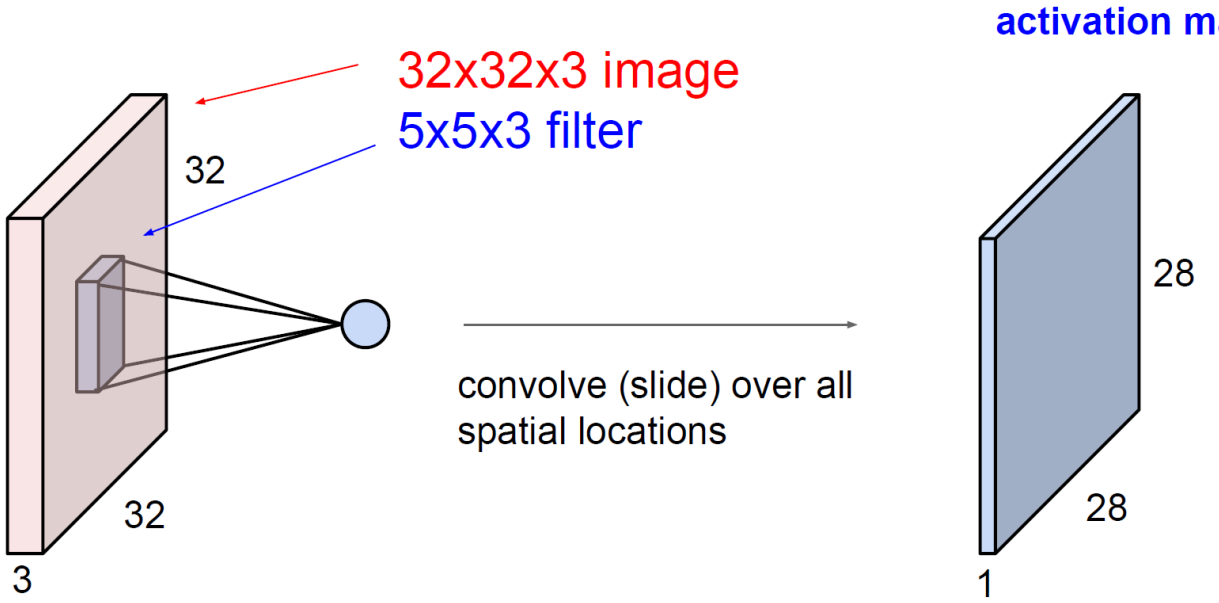
0	0.2	0	
0.2	0.4	0.2	
0	0.2	0	

3x3 filter

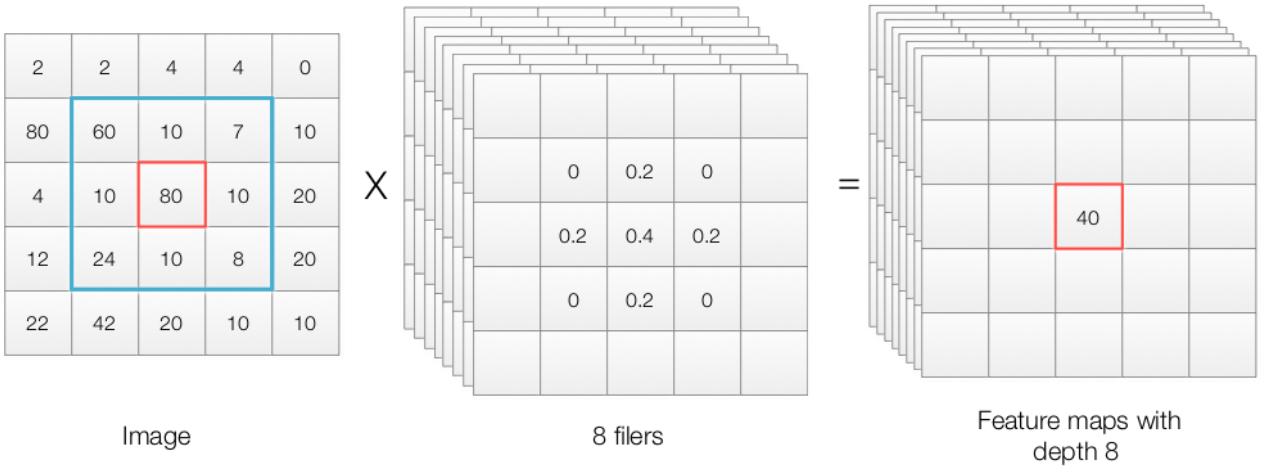
=

		40		

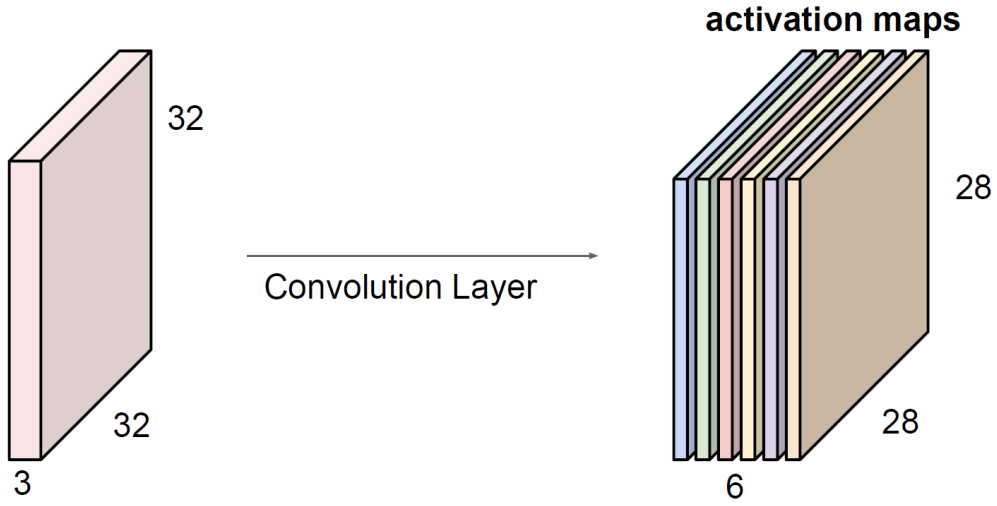
80x0.4 + 4x0.2x10=40



Konvolucioni sloj



For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:



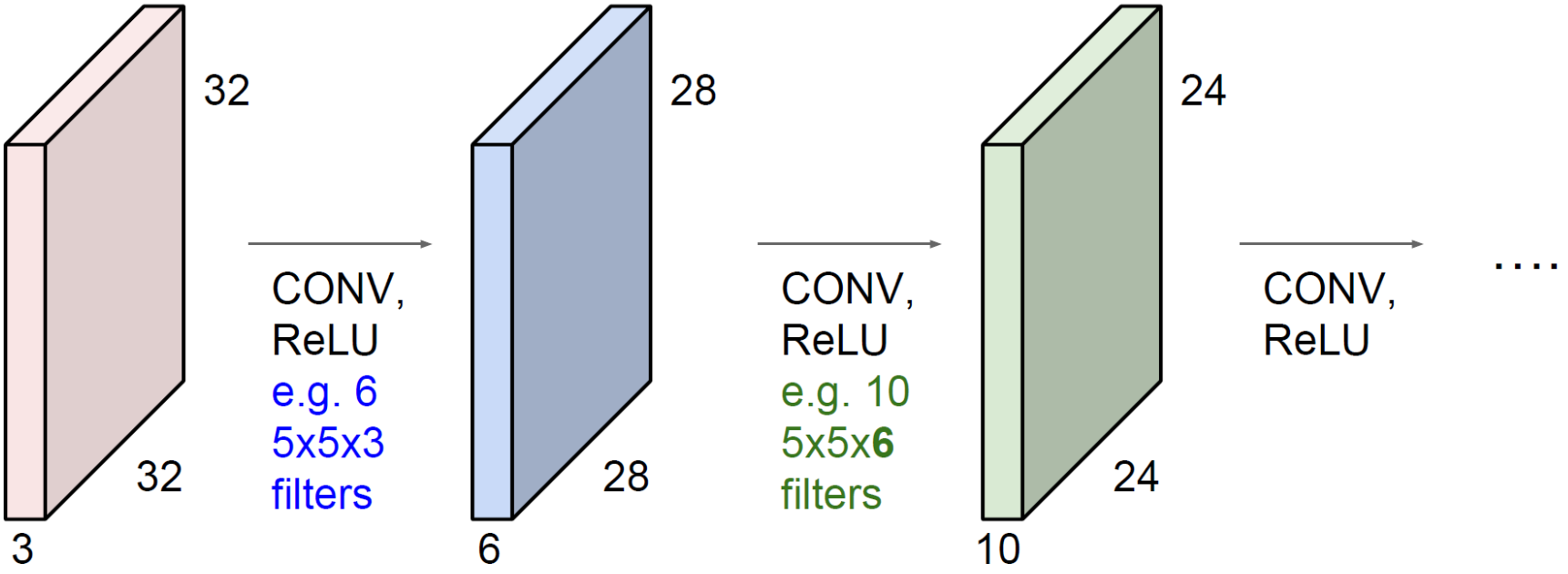
We stack these up to get a "new image" of size 28x28x6!

Možemo imati više filtera

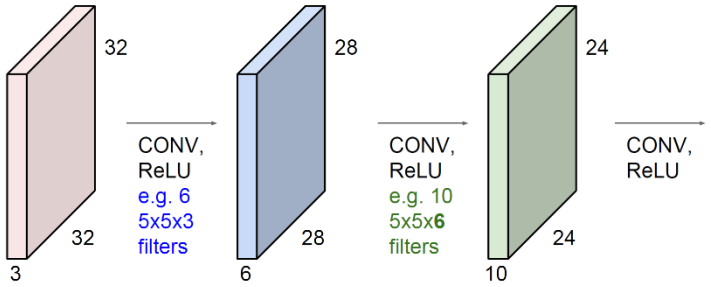
- Svaki rezultuje jednom aktivacionom mapom
- Svaki traži određeni koncept (šablon) u ulazima

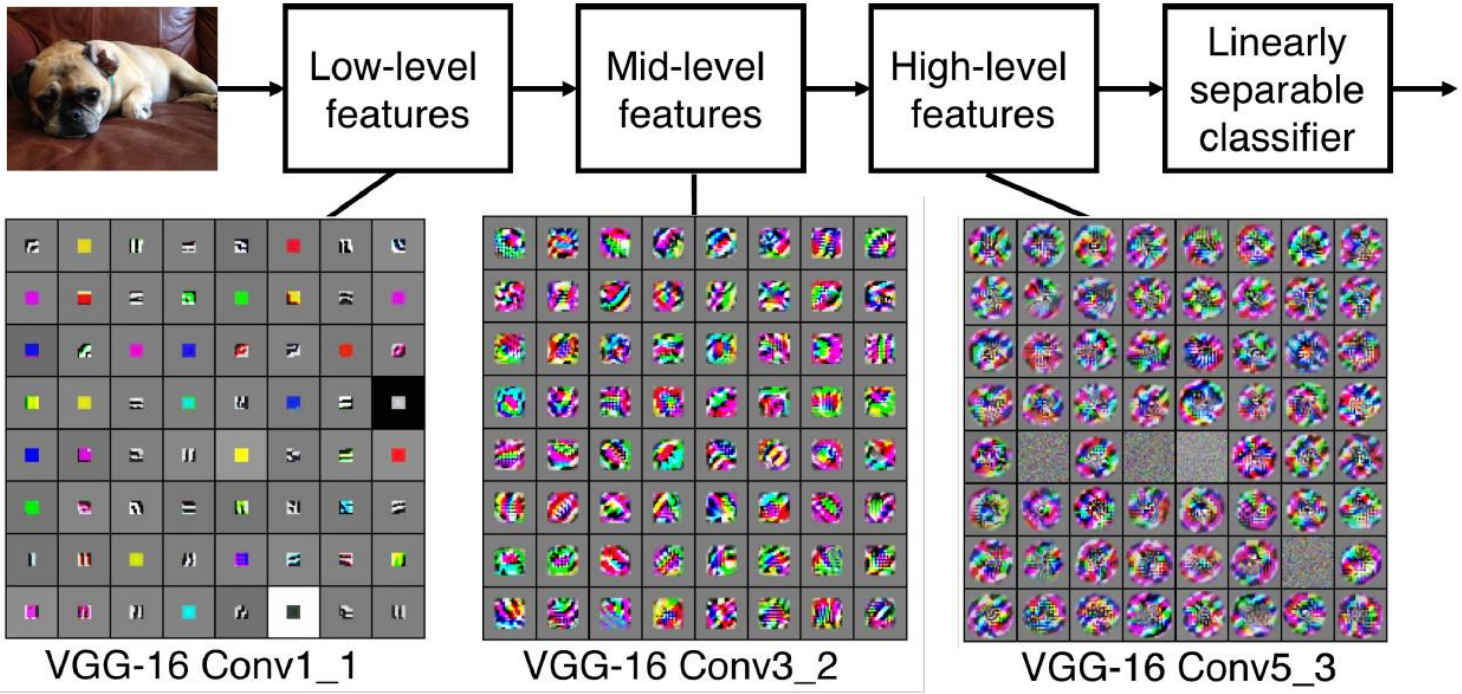
Više konvolucionih slojeva

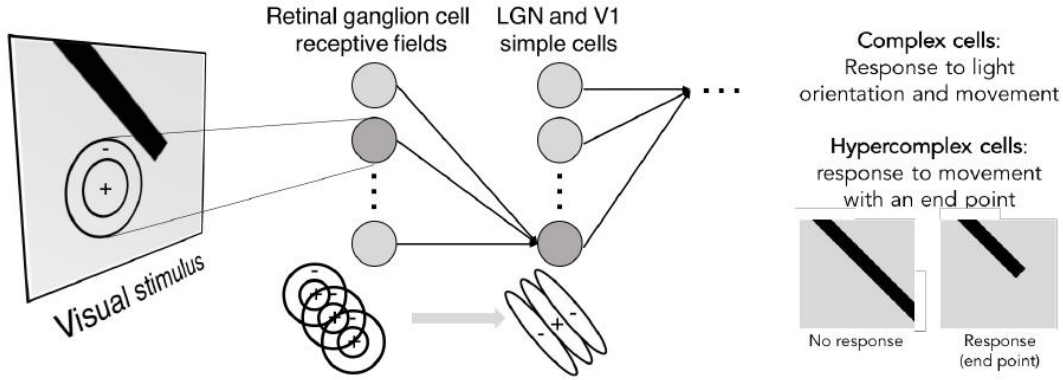
Preview: ConvNet is a sequence of Convolutional Layers, interspersed with activation functions



Hijerarhijska organizacija



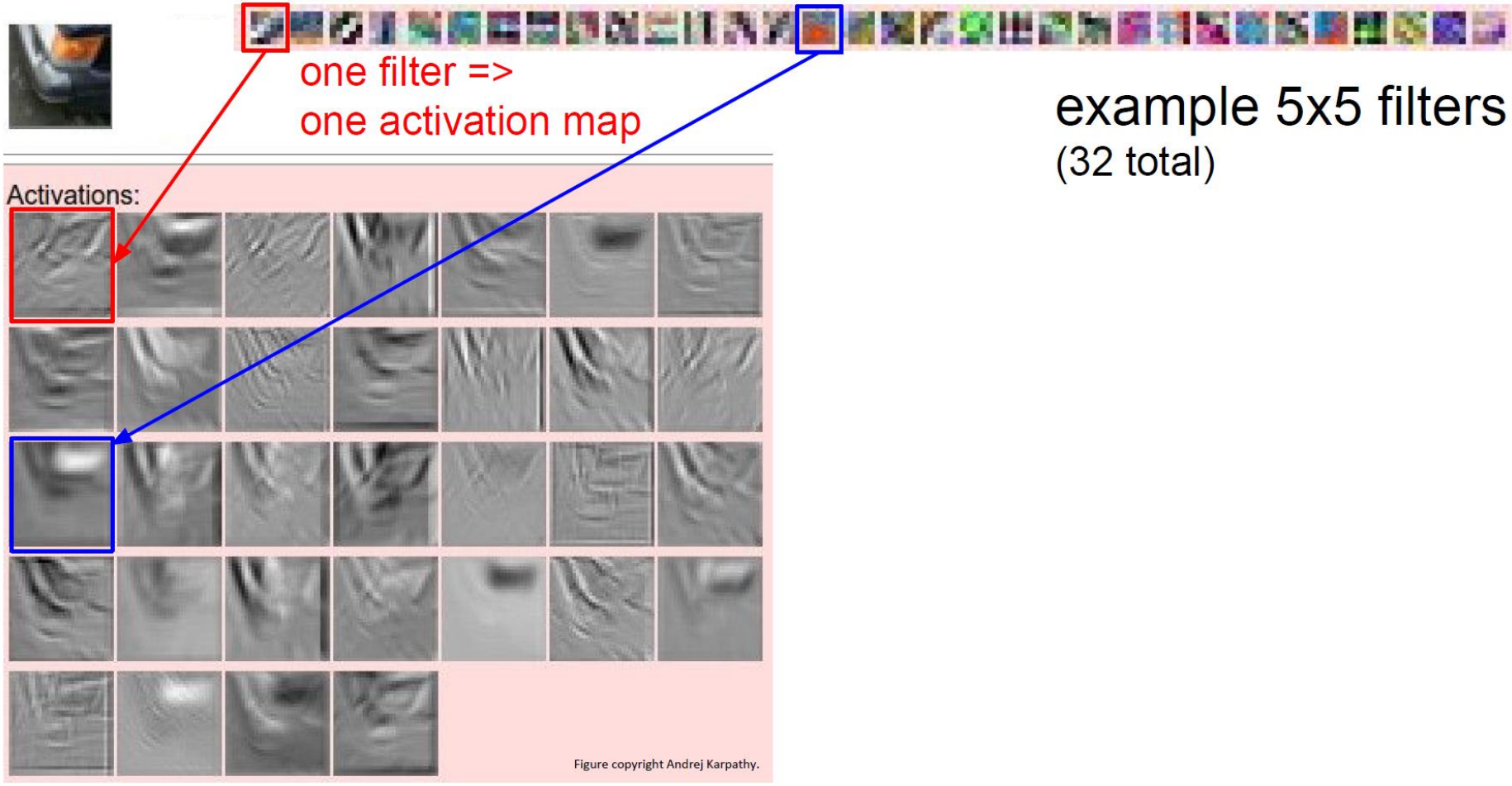




Vizuelizacija

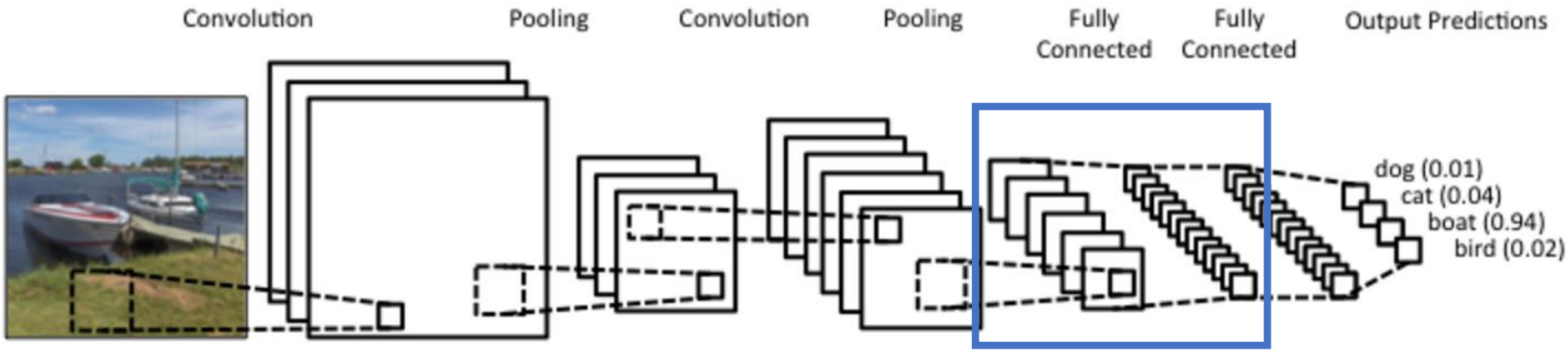
Deluje da traži dijagonalno orijentisane ivice

→ jake aktivacije (belo) tamo gde je prisutan ovaj tip ivica



Primeri aktivacionih mapa koje proizvodi svaki filter (prikazani gore)

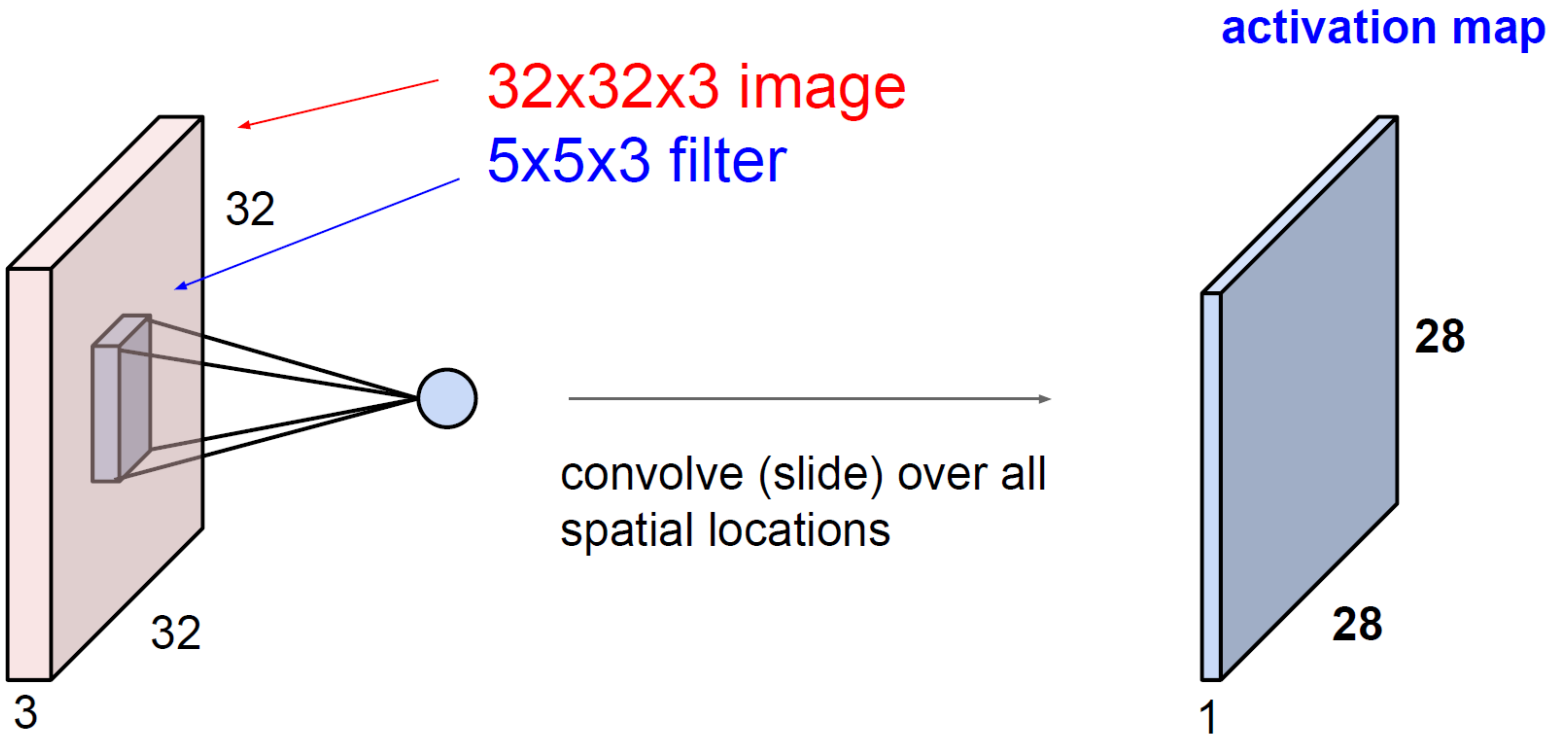
Arhitektura CNN




```
    model.add(MaxPool2D((2, 2), strides=2, padding="same"))
    model.add(Flatten())
    model.add(Dense(units=512, activation="relu"))
```

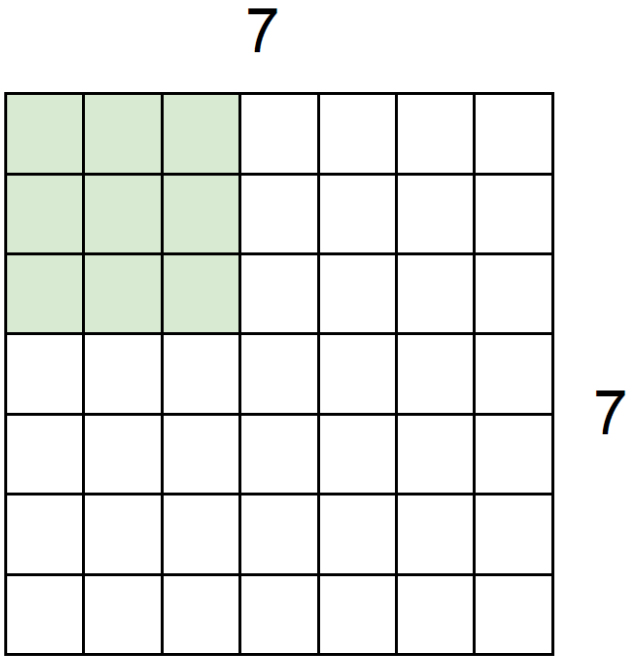
Prostorne dimenzije

A closer look at spatial dimensions:



Prostorne dimenzije

A closer look at spatial dimensions:

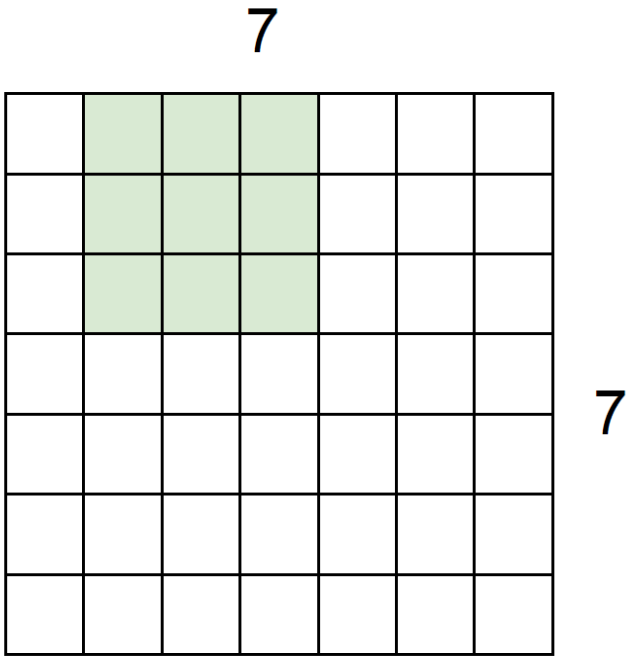


7x7 input (spatially)
assume 3x3 filter

Pomeraj (*stride*) = 1

Prostorne dimenzije

A closer look at spatial dimensions:

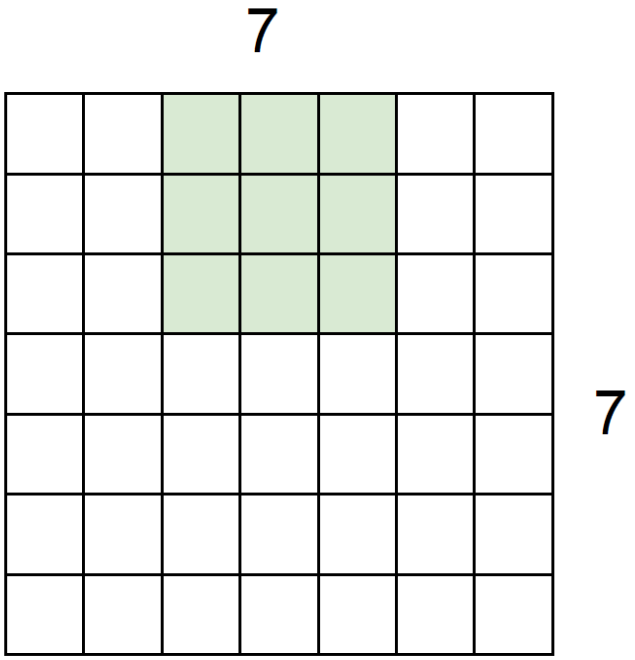


7x7 input (spatially)
assume 3x3 filter

Pomeraj (*stride*) = 1

Prostorne dimenzije

A closer look at spatial dimensions:

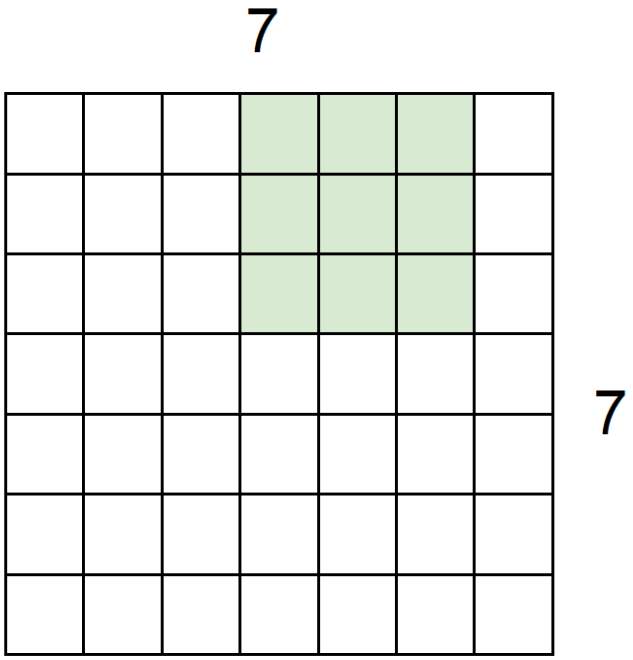


7x7 input (spatially)
assume 3x3 filter

Pomeraj (*stride*) = 1

Prostorne dimenzije

A closer look at spatial dimensions:

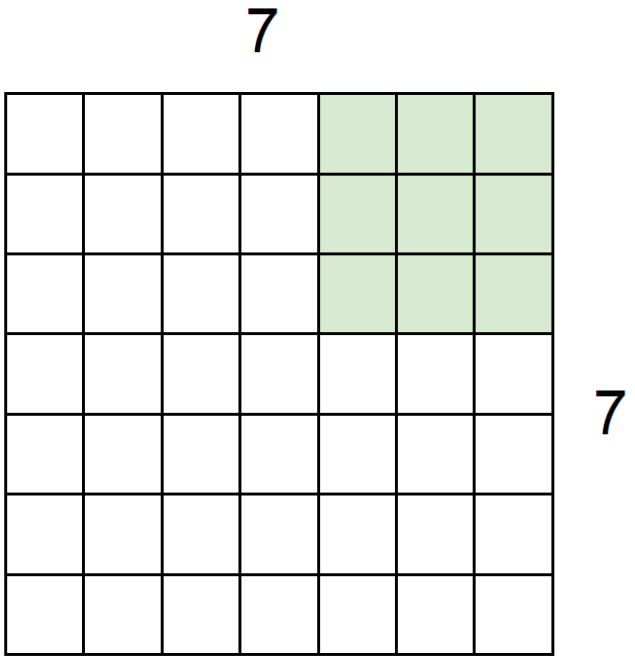


7x7 input (spatially)
assume 3x3 filter

Pomeraj (*stride*) = 1

Prostorne dimenzije

A closer look at spatial dimensions:



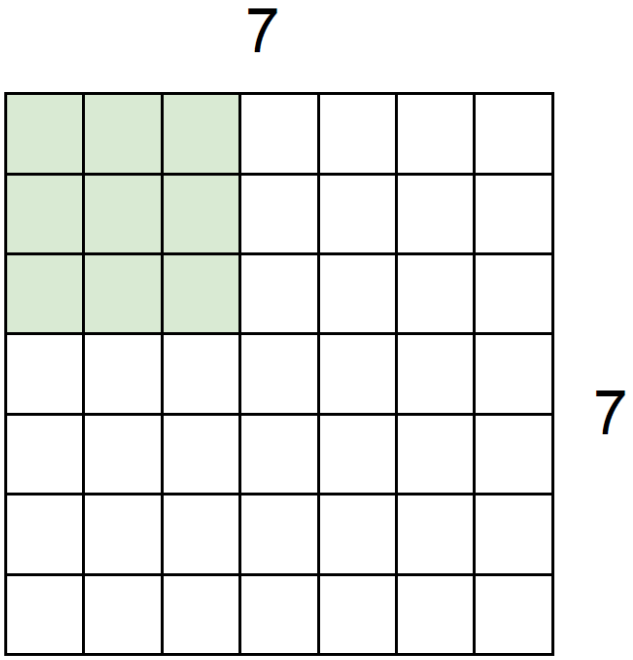
7x7 input (spatially)
assume 3x3 filter

=> 5x5 output

Pomeraj (*stride*) = 1

Prostorne dimenzije

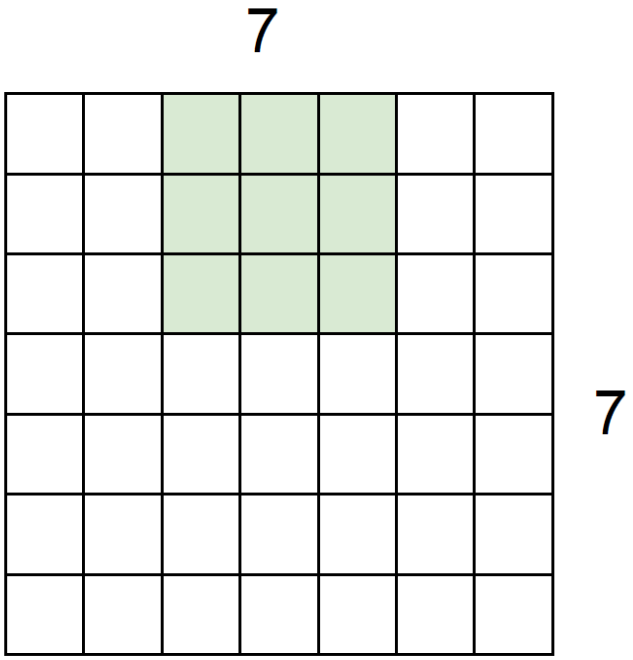
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

Prostorne dimenzije

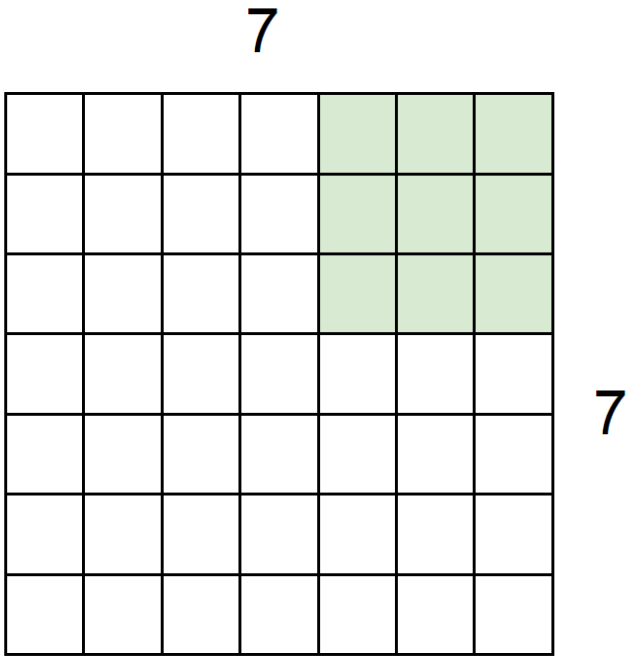
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

Prostorne dimenzije

A closer look at spatial dimensions:



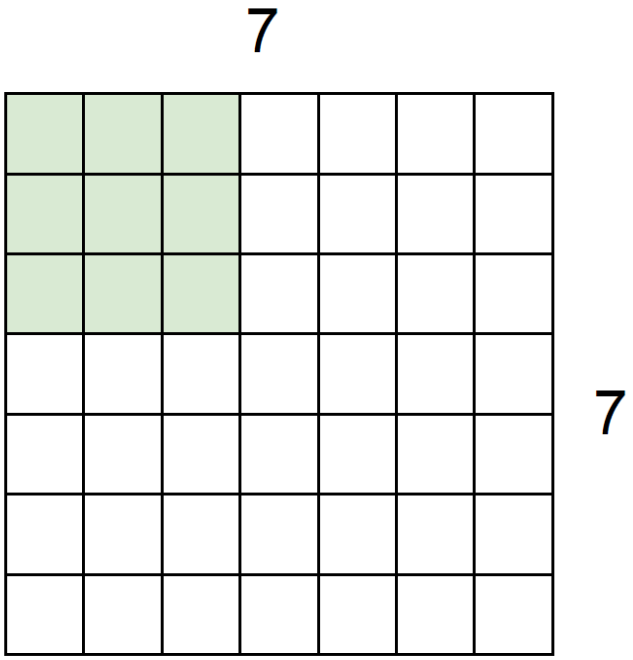
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**
=> 3x3 output!

Prostorne dimenzije rezultujućih aktivacija zavise od:

- Veličine filtera
- Pomeraja filtera

Prostorne dimenzije

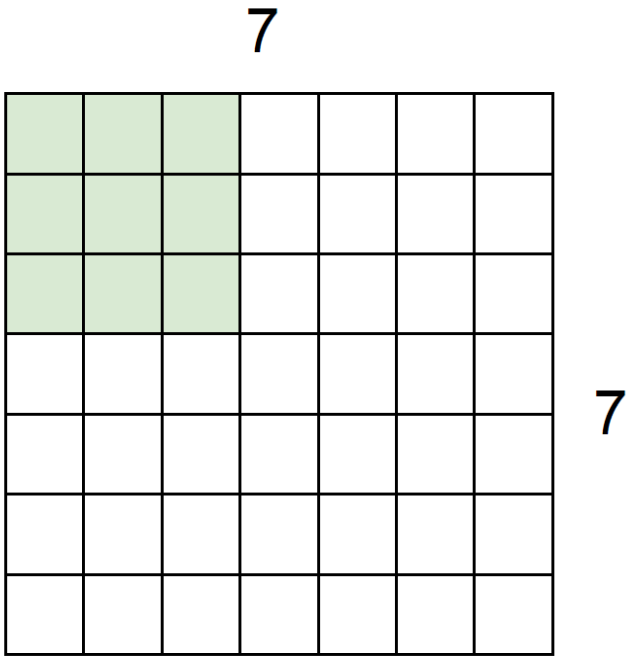
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

Prostorne dimenzije

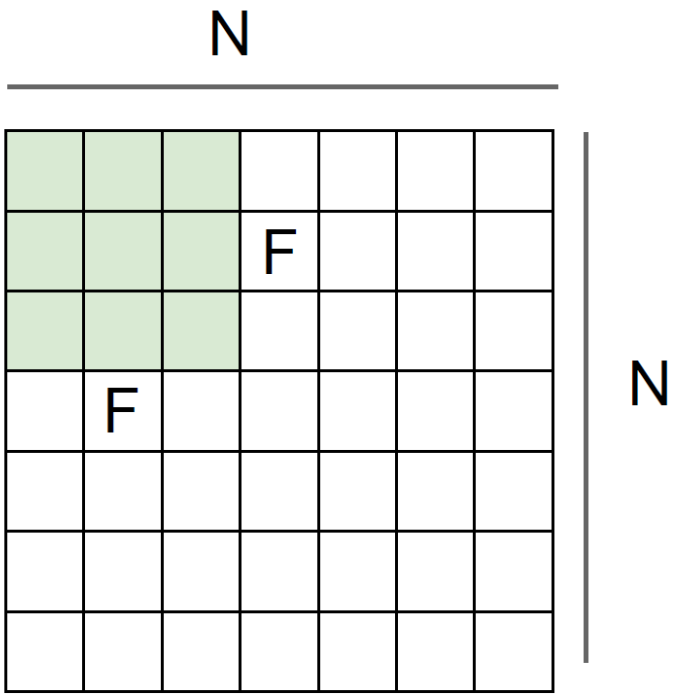
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

doesn't fit!
cannot apply 3x3 filter on
7x7 input with stride 3.

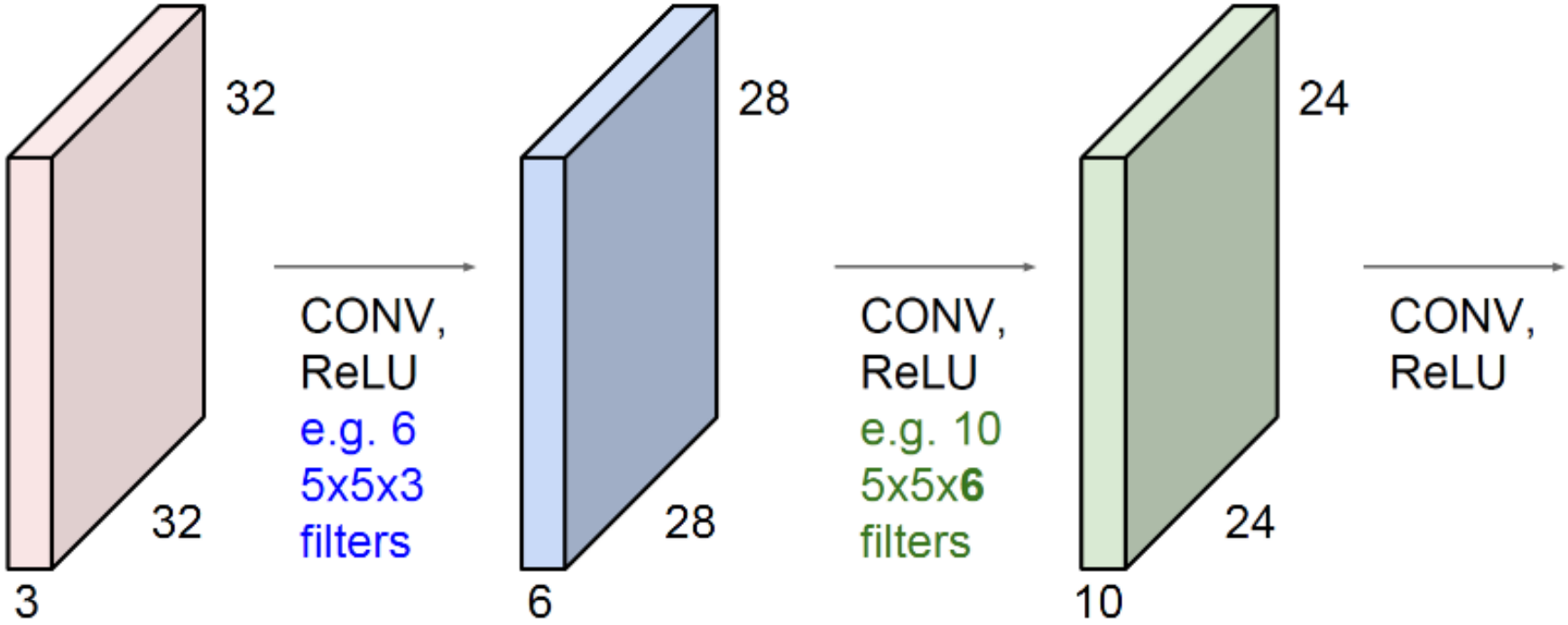
Prostorne dimenzije



Output size:
(N - F) / stride + 1

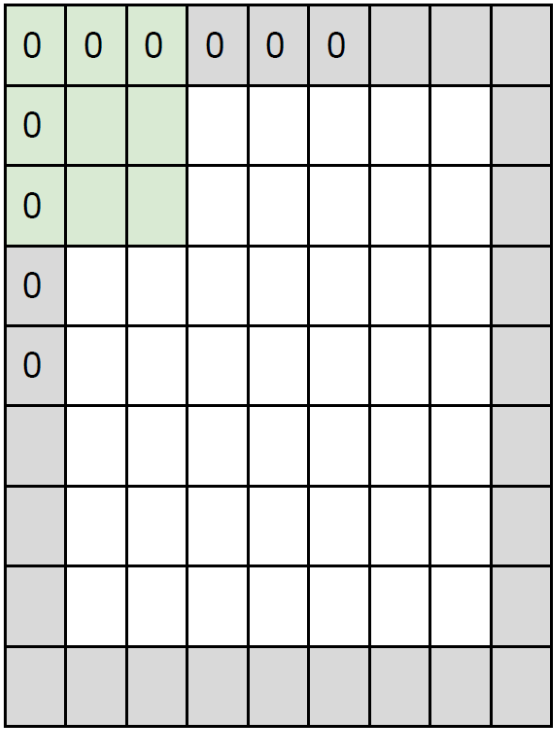
e.g. N = 7, F = 3:
stride 1 => (7 - 3)/1 + 1 = 5
stride 2 => (7 - 3)/2 + 1 = 3
stride 3 => (7 - 3)/3 + 1 = 2.33 :\

Prostorne dimenzije



Proširivanje ivica nulama (*zero padding*)

In practice: Common to zero pad the border



e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

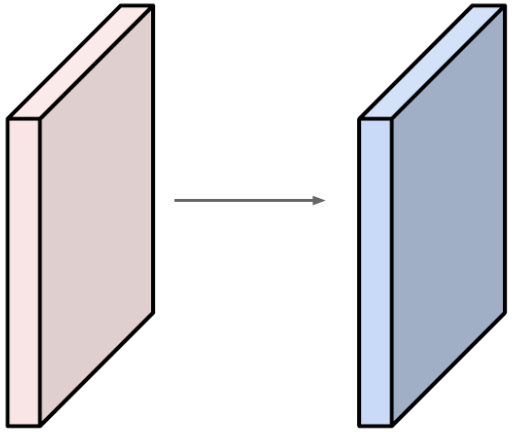
7x7 output!
in general, common to see CONV layers with stride 1, filters of size FxF, and zero-padding with (F-1)/2. (will preserve size spatially)
e.g. F = 3 => zero pad with 1
F = 5 => zero pad with 2
F = 7 => zero pad with 3

Hiperparametri

- Veličina filtera
  - Obično se koriste filteri $3 \times 3, 5 \times 5, 7 \times 7$
- *Zero padding*
  - Cilj je da veličina izlaza bude jednaka veličini ulaza
    - Za 3×3, bira se 1
    - Za 5×5, bira se 2
    - …
  - Bez *zero padding*, veličina aktivacionih mapa bi se brzo smanjivala sa brojem slojeva, što je nezgodno ako imamo zaista duboke mreže
- Veličina pomeraja

Prostorne dimenzije

Examples time:

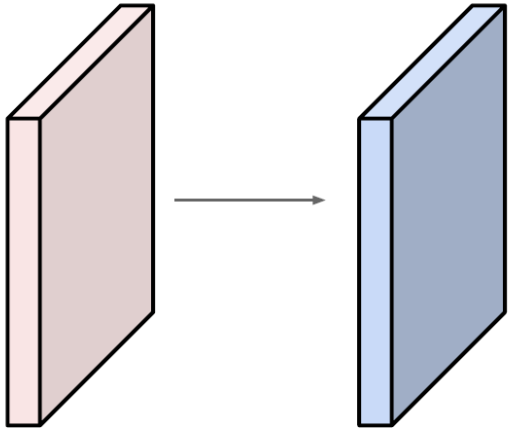


Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Output volume size:
(32+2*2-5)/1+1 = 32 spatially, so
32x32x10

Broj parametara

Examples time:

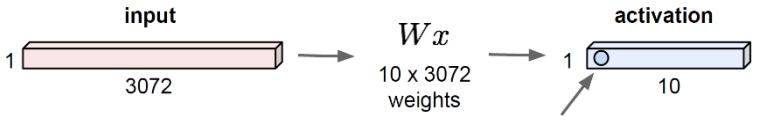


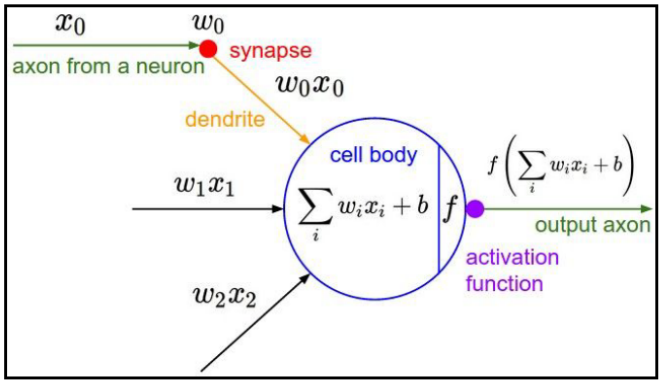
Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Number of parameters in this layer?
each filter has 5*5*3 + 1 = 76 params (+1 for bias)
=> 76*10 = **760**

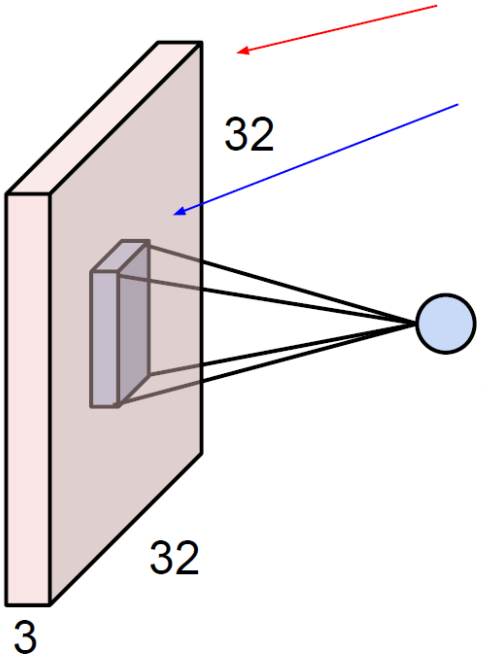
Neuron u CNN

32x32x3 image -> stretch to 3072 x 1





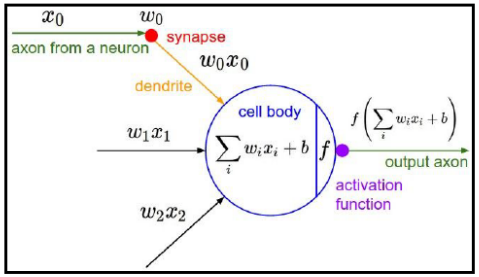
It's just a neuron with local connectivity...

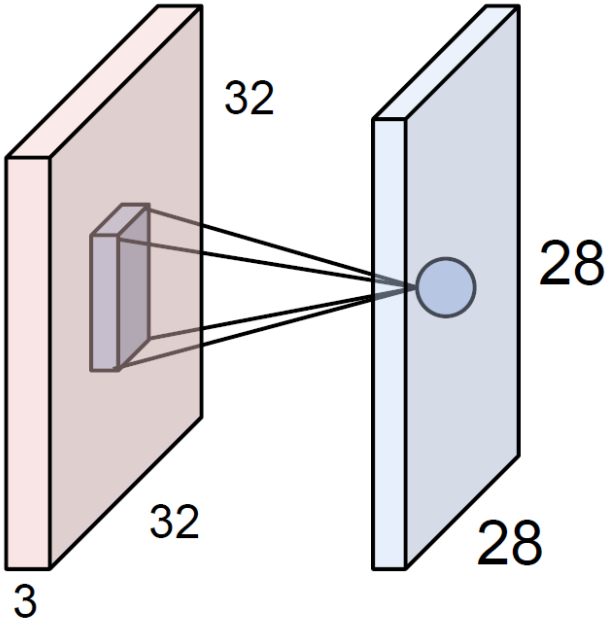


1 number:
the result of taking a dot product between the filter and this part of the image
(i.e. 5*5*3 = 75-dimensional dot product)

Neuron u CNN

The brain/neuron view of CONV Layer





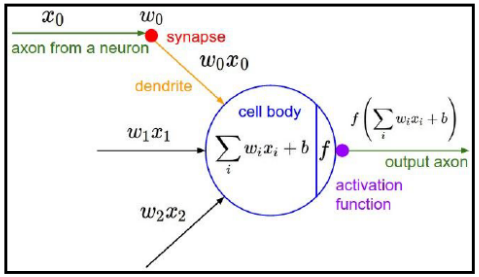
An activation map is a 28x28 sheet of neuron outputs:

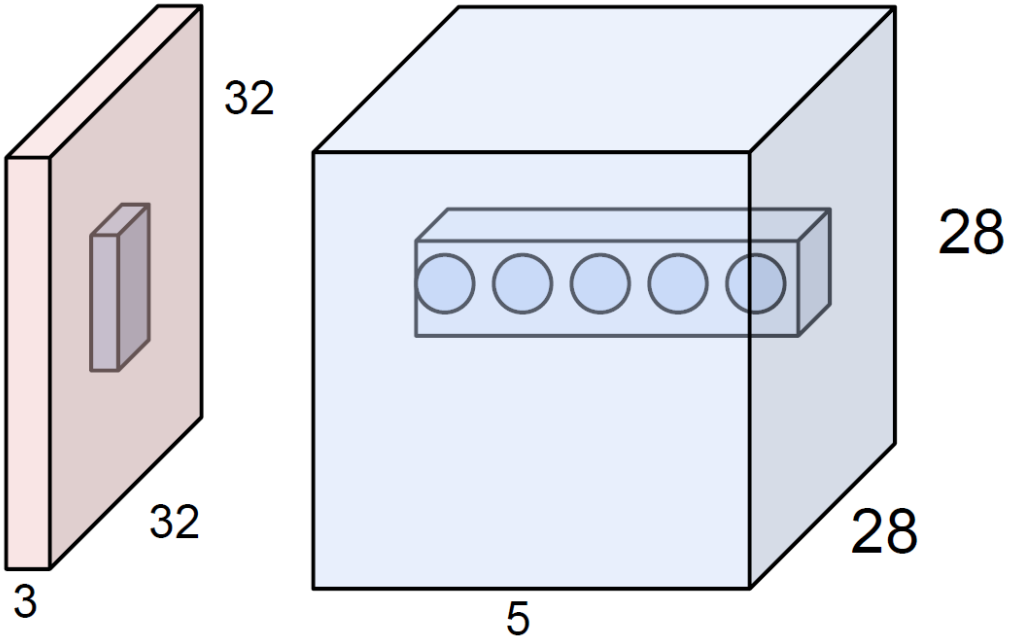
1. Each is connected to a small region in the input
2. All of them share parameters

"5x5 filter" -> "5x5 receptive field for each neuron"

Neuron u CNN

The brain/neuron view of CONV Layer

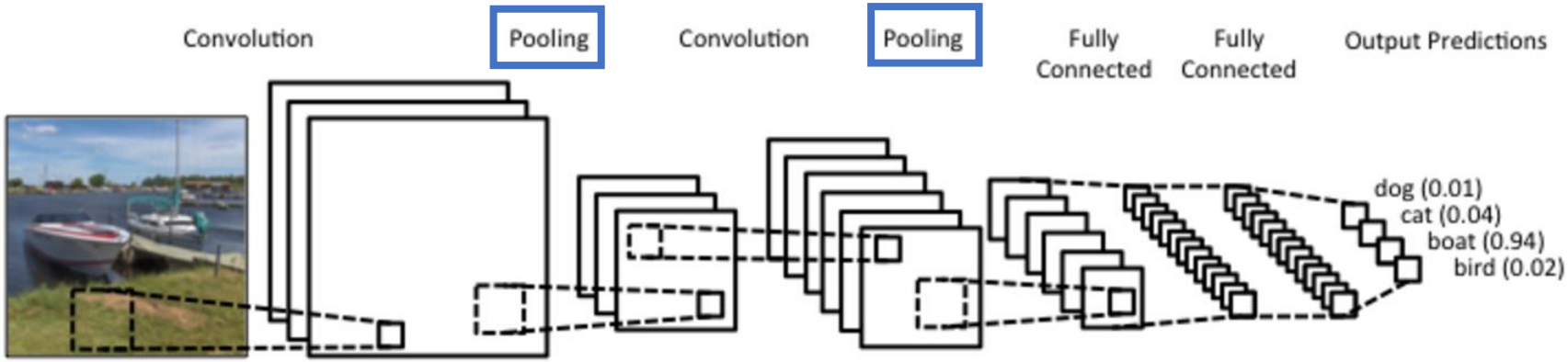




E.g. with 5 filters,
CONV layer consists of neurons arranged in a 3D grid (28x28x5)

There will be 5 different neurons all looking at the same region in the input volume

Slojevi sažimanja (*pooling*)

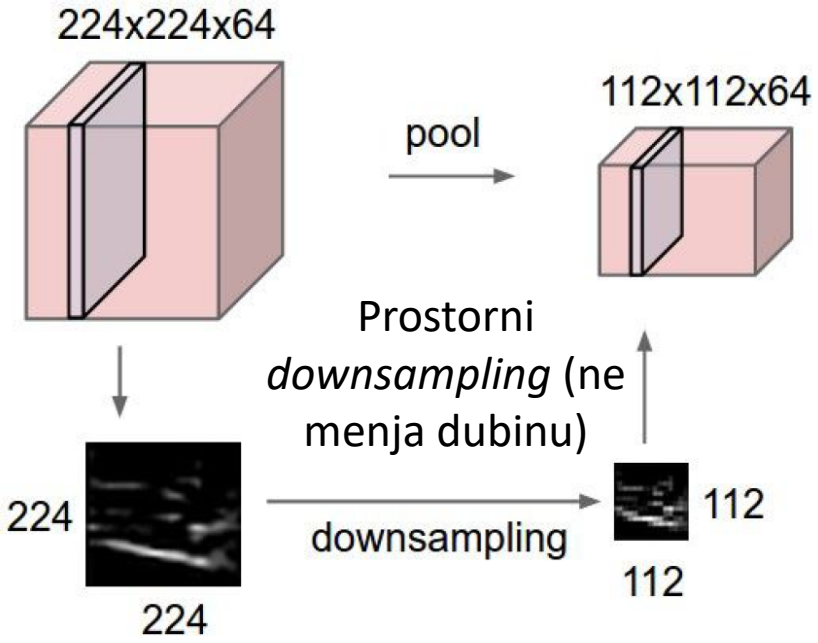


Slojevi sažimanja (*pooling*)

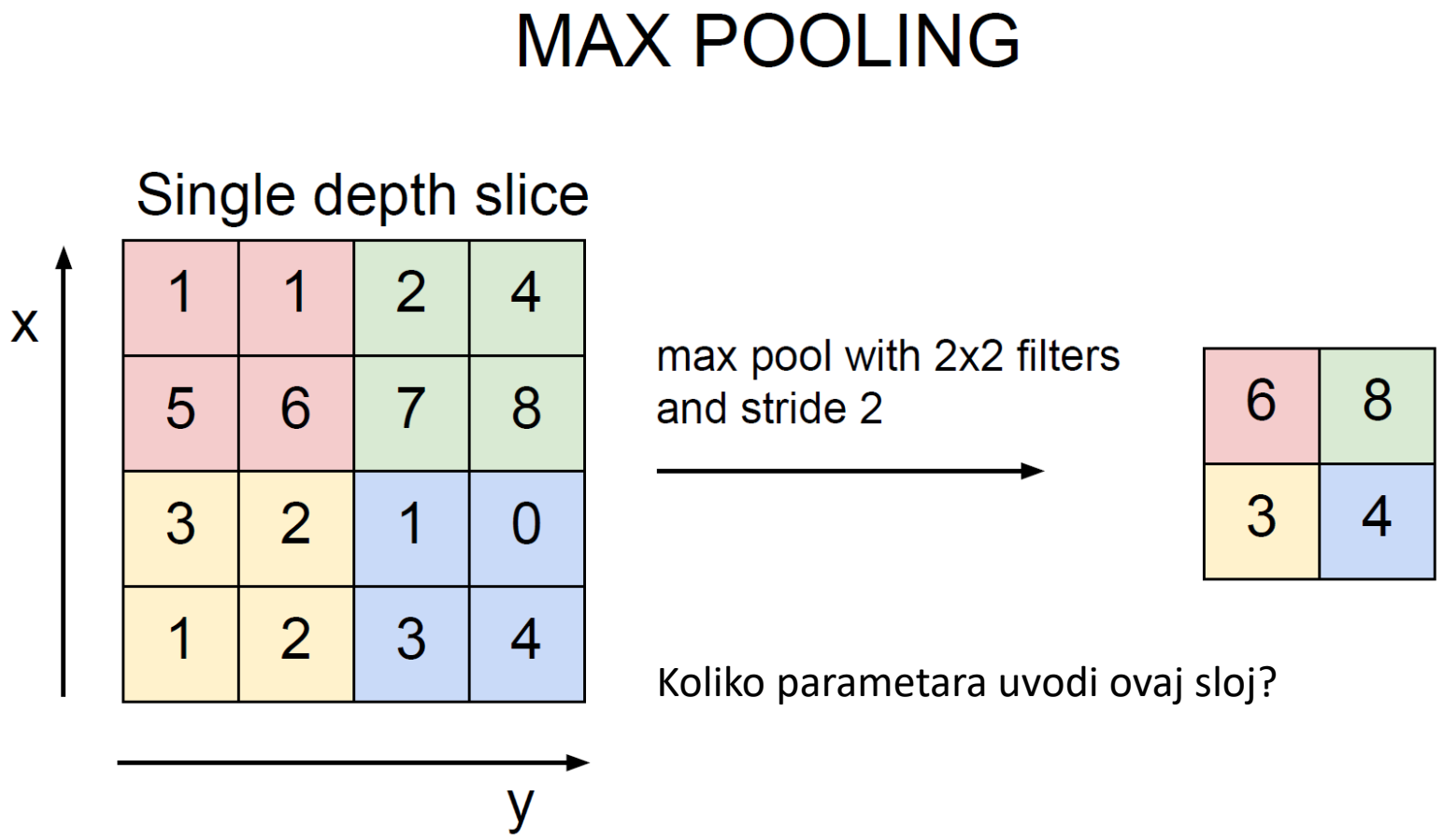
Pooling layer

- makes the representations smaller and more manageable →
- operates over each activation map independently:

- Smanjuju broj računanja
- Invarijantnost u nekoj regiji

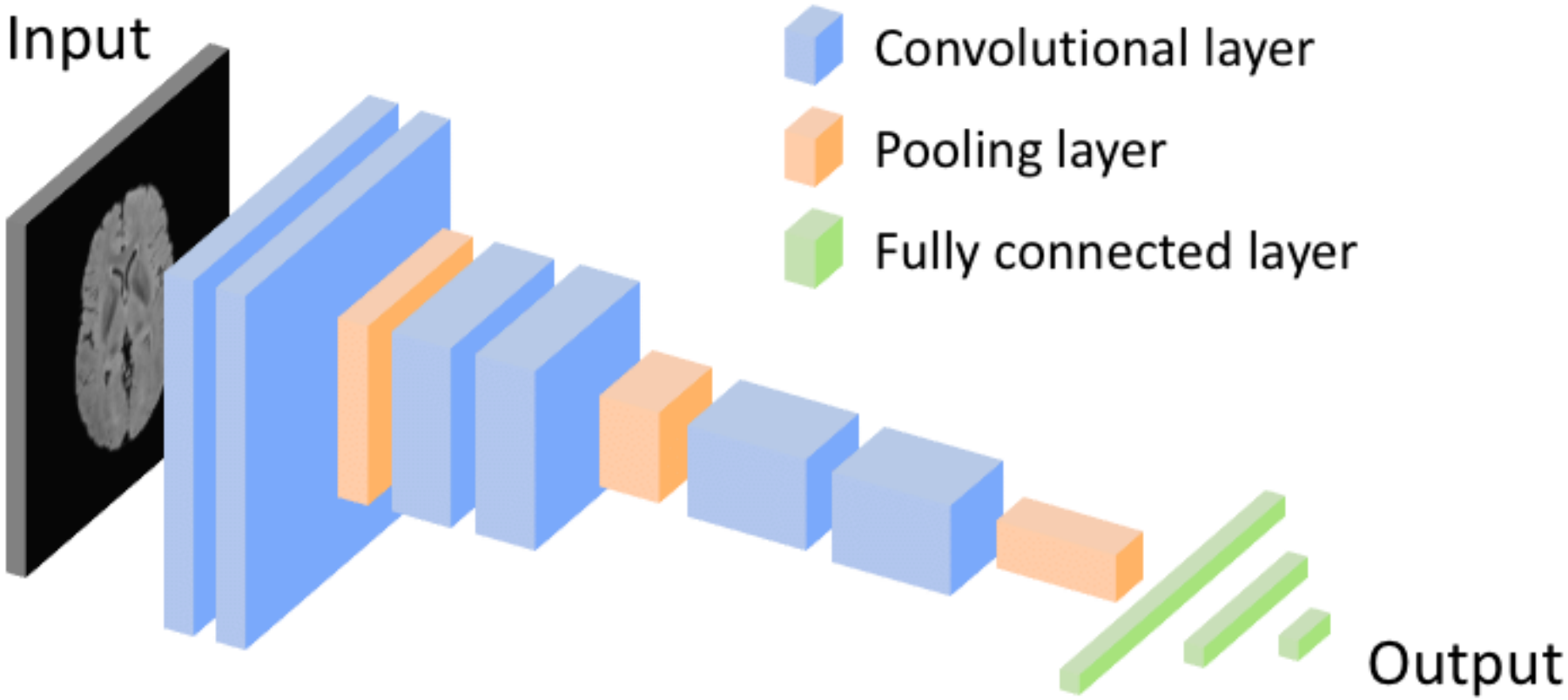


Često korišćen: *Max Pooling*



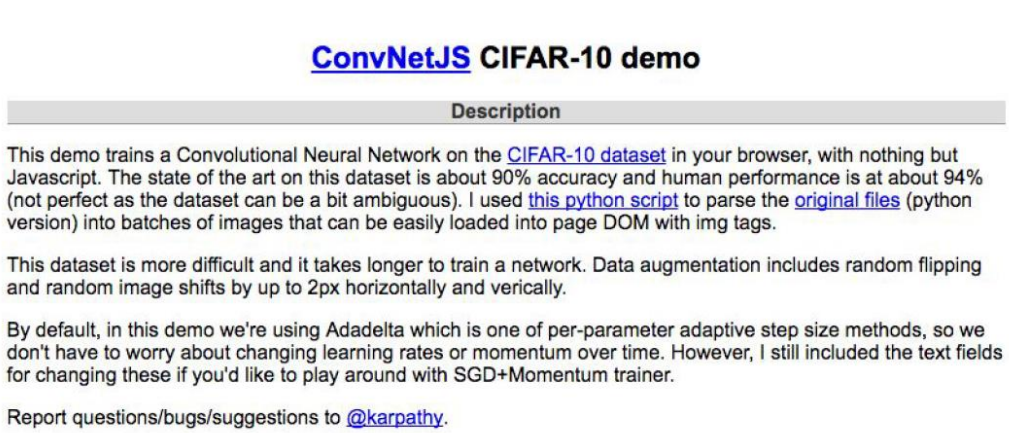
Koliko parametara uvodi ovaj sloj?

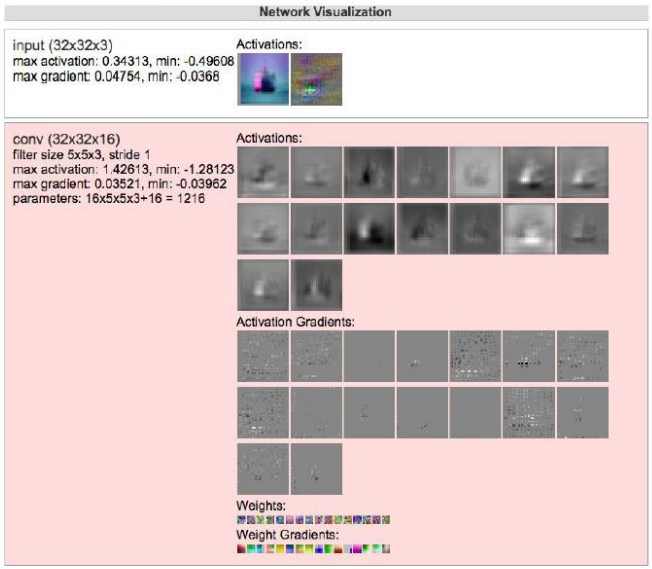
Potpuno povezan sloj



Demo

[ConvNetJS demo: training on CIFAR-10]





http://cs.stanford.edu/people/karpathy/convnetjs/demo/cifar10.html

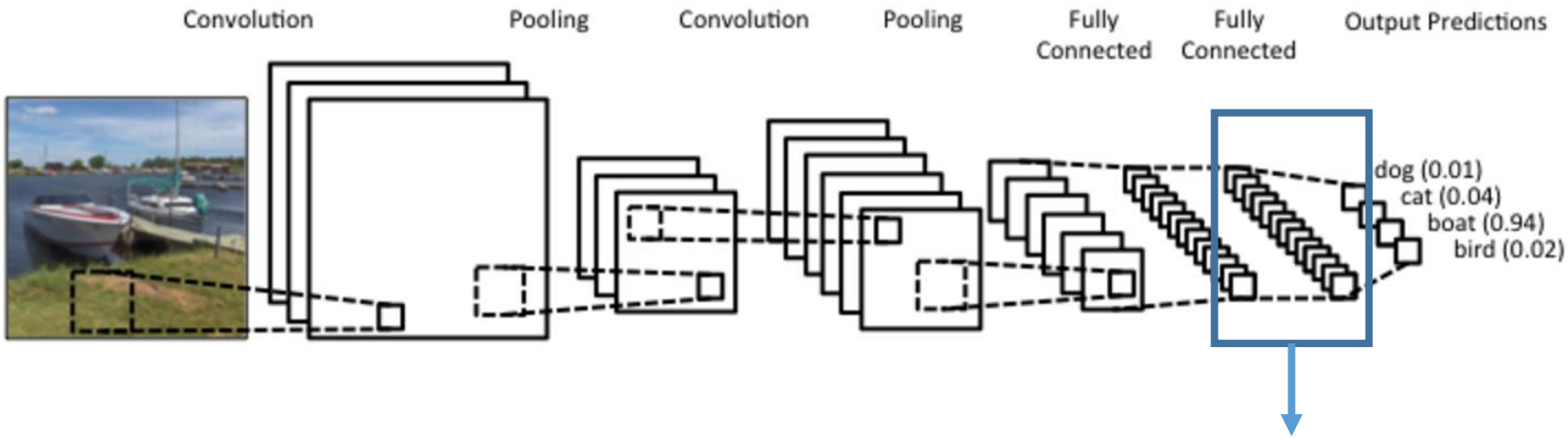
Transfer learning

Transfer Learning

"You need a lot of a data if you want to train/use CNNs"

BUSTED

Transfer learning



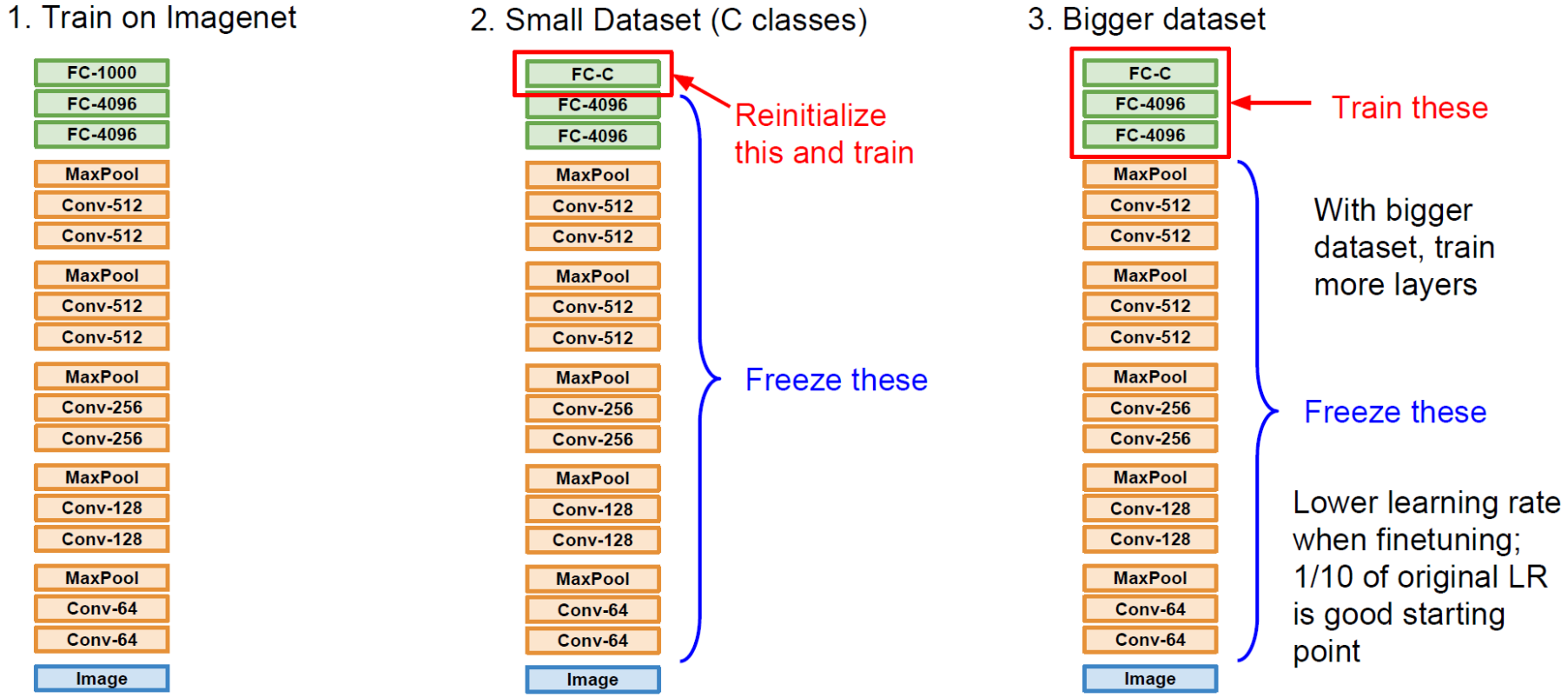
Reprezentacija slike
u, npr., 4096 obeležja

(tačan broj „obeležja“
zavisi od arhitekture)

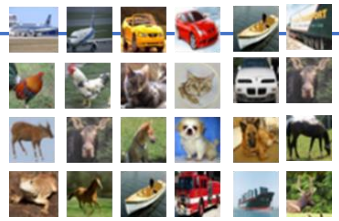
Transfer learning

Transfer Learning with CNNs

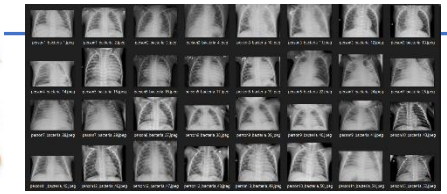
Donahue et al, "DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition", ICML 2014
Razavian et al, "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition", CVPR Workshops 2014

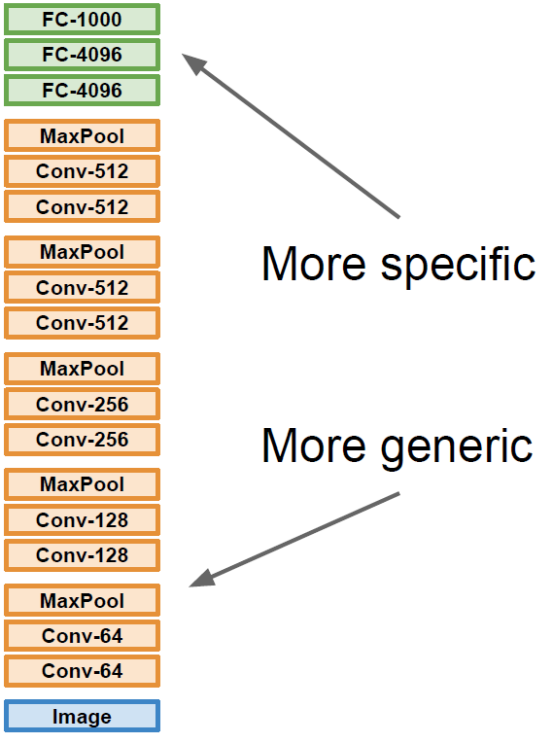


Transfer learning





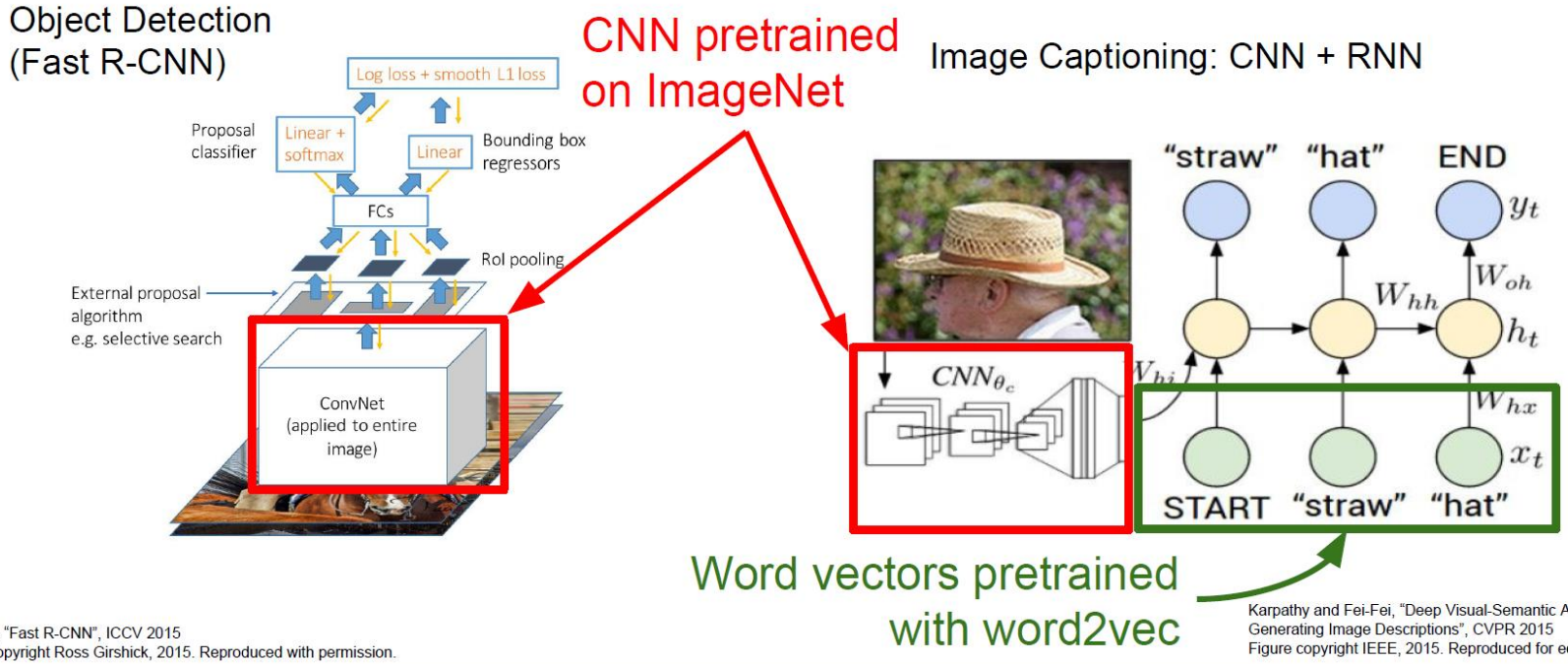




	very similar dataset	very different dataset
very little data	Use Linear Classifier on top layer	You're in trouble… Try linear classifier from different stages
quite a lot of data	Finetune a few layers	Finetune a larger number of layers

Transfer learning

Transfer learning with CNNs is pervasive…
(it's the norm, not an exception)



Girshick, "Fast R-CNN", ICCV 2015
Figure copyright Ross Girshick, 2015. Reproduced with permission.

Karpathy and Fei-Fei, "Deep Visual-Semantic Alignments for Generating Image Descriptions", CVPR 2015
Figure copyright IEEE, 2015. Reproduced for educational purposes.

Transfer learning

Takeaway for your projects and beyond:

Have some dataset of interest but it has < ~1M images?

1. Find a very large dataset that has similar data, train a big ConvNet there
2. Transfer learn to your dataset

Deep learning frameworks provide a "Model Zoo" of pretrained models so you don't need to train your own

Caffe: https://github.com/BVLC/caffe/wiki/Model-Zoo
TensorFlow: https://github.com/tensorflow/models
PyTorch: https://github.com/pytorch/vision

Klasifikacija slika pomoću NN – ASL




```
[ ] train_df.head()
```

	label	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	pixel8	pixel9	...	pixel775	pixel776	pixel777	pixel778	pixel779	pixel780	pixel781	pixel782	pixel783	pixel784
0	3	107	118	127	134	139	143	146	150	153	...	207	207	207	207	206	206	206	204	203	202
1	6	155	157	156	156	156	157	156	158	1											
2	2	187	188	188	187	187	186	187	188	1											
3	2	211	211	212	212	211	210	211	210	2											
4	12	164	167	170	172	176	179	180	184	1											

5 rows × 785 columns

```
[ ] from tensorflow.keras.models import Sequential
    from tensorflow.keras.layers import Dense

    model = Sequential()
    model.add(Dense(units=512, activation='relu', input_shape=(784,)))
    model.add(Dense(units=512, activation='relu'))
    model.add(Dense(units=num_classes, activation='softmax'))
```

Pretprocesiranje:

- normalizacija
- enkodiranje labela

```
[ ] model.summary()
```

```
Model: "sequential_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 dense_3 (Dense)             (None, 512)               401920

 dense_4 (Dense)             (None, 512)               262656

 dense_5 (Dense)             (None, 24)                12312

=================================================================
Total params: 676888 (2.58 MB)
Trainable params: 676888 (2.58 MB)
Non-trainable params: 0 (0.00 Byte)
```

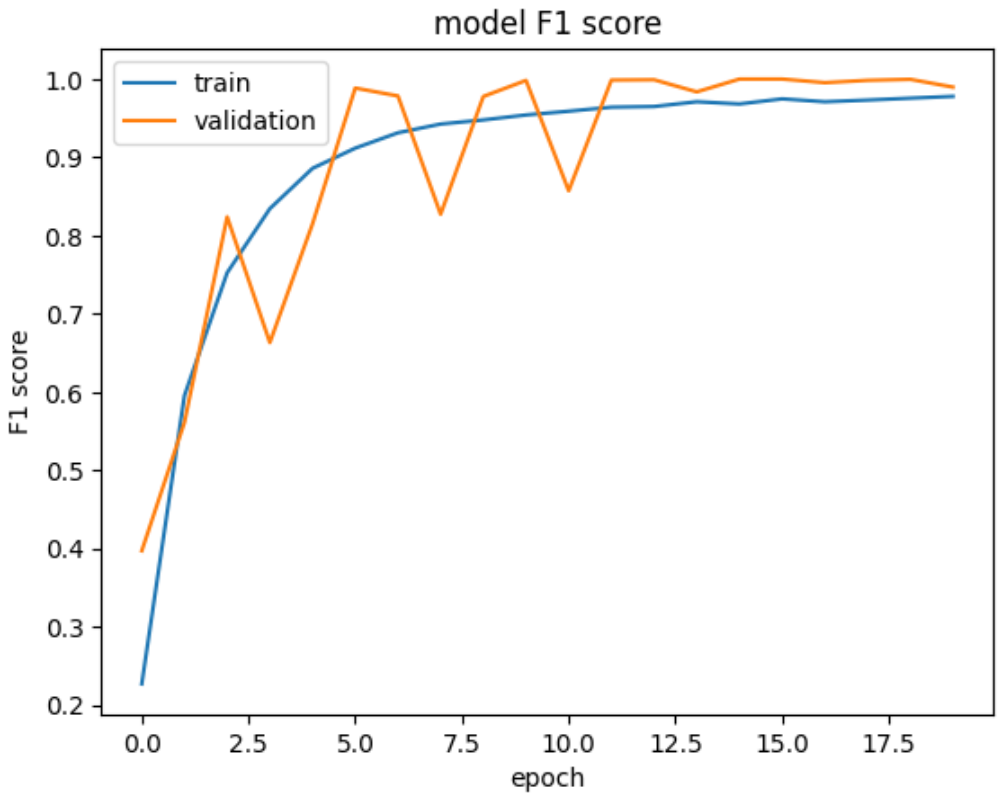
Klasifikacija slika pomoću NN – ASL

```
[ ]  import pandas as pd

     train_df = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/CNN vežbe/data/asl_data/sign_mnist_train.csv")
     test_df = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/CNN vežbe/data/asl_data/sign_mnist_valid.csv")
```

```
[ ]  from sklearn.model_selection import train_test_split

     x_train, x_val, y_train, y_val = train_test_split(x_train, y_train, test_size=0.2, stratify=y_train, random_state=42)
```



F1 Score on Test Data:
0.79

ASL

	F1-score of test	No. trainable params
HOG + SVM	0.63	
Feedforward fully-connected NN	0.79	676 888 (2.58 MB)
CNN	0.95	263 749 (1.01 MB)
CNN + augmentacija	1.00	263 749 (1.01 MB)

